

Алматы қаласының Білім басқармасы
«ALMATY POLYTECHNIC COLLEGE» КМҚК

ҚОРҒАУҒА ЖІБЕРІЛЕДІ
Директордың оқу ісі жөніндегі орынбасары
_____ М.Мәлікова
«01» ақпан 2026 жыл

ДИПЛОМДЫҚ ЖОБА

**тақырыбы: «ТӘРБИЕ САҒАТЫН ЭЛЕКТРОНДЫ ТҮРДЕ БЕКІТУ
ЖӘНЕ БАСҚАРУҒА АРНАЛҒАН АҚПАРАТТЫҚ ЖҮЙЕ
ӨЗІРЛЕУ»**

ТҮСІНДІРМЕ ЖАЗБА
ҚР АРС 06130100 ДЖ ТХ

Мамандығы: 06130100 «Бағдарламалық қамтамасыз ету (түрлері бойынша)»
Біліктілігі: 4S06130105 «Ақпараттық жүйелер технигі»
Орындаған: П22-4Б тобының студенті Бахитияр Санжар
Жетекшісі: Қали Набат

Алматы 2026

Бекітемін
Директордың оқу ісі жөніндегі орынбасары
_____ М.Мәлікова
«25» желтоқсан 2026 жыл

06130100 «Бағдарламалық қамтамасыз ету (түрлері бойынша)»
мамандығы, 4S06130105 «Ақпараттық жүйелер технигі» біліктілігі бойынша 4
курс П22-4Б тобының студенті Бахитияр Санжар дипломдық жобасына

ТАПСЫРМА

Дипломдық жоба тақырыбы: «Тәрбие сағатын электронды түрде бекіту
және басқаруға арналған ақпараттық жүйе әзірлеу».

Колледж бойынша бекітілген бұйрық №

Дипломдық	жобаны	тапсыру	мерзімі
-----------	--------	---------	---------

Дипломдық жобада қарастырылатын сұрақтар тізімі:

- тәрбие сағатын жоспарлау, бекіту және есепке алу процесін талдау;
- әкімші, куратор және студент рөлдеріне арналған талаптарды анықтау;
- электронды бекіту, Word-құжат жүктеу және электронды қолтаңба қою процесін жобалау;
- Cloudflare Workers, Nono, D1, KV және Queues негізіндегі серверлік бөлікті сипаттау;
- React, Vite және Tailwind CSS негізіндегі веб-интерфейсті жобалау;
- Telegram хабарламалары, AI-көмекші және есеп беру модульдерін қарастыру;
- деректер қорының құрылымын, API маршруттарын және қауіпсіздік механизмдерін талдау;
- тестілеу нәтижелерін, экономикалық есепті және еңбек қауіпсіздігі талаптарын көрсету.

Графикалық материал тізімі:

- ақпараттық жүйенің жалпы архитектуралық схемасы;
- тәрбие сағатын бекіту процесінің алгоритмі;
- деректер қорының ER-диаграммасы;
- пайдаланушы рөлдері бойынша use-case диаграммасы;
- негізгі интерфейс экрандарының скриншоттары;
- API маршруттары мен модульдерінің кестесі.

Дипломдық жоба жетекшісі _____ Қали Набат
«Бағдарламалық қамтамасыз ету» бөлім отырысында қаралды
«26» қазан 2026 ж. Хаттама № 3
Бөлім меңгерушісі _____ У.Саурбек

МАЗМҰНЫ

КІРІСПЕ.....	5
1 ЖАЛПЫ БӨЛІМ.....	7
1.1 Бағдарламалық өнімнің сипаттамасы.....	7
1.2 Пәндік аймақты талдау және қолданыстағы шешімдерге шолу.....	7
1.3 Қолданылған технологиялар мен тілдер.....	8
1.4 Жүйеге қойылатын талаптар.....	10
2 АРНАЙЫ БӨЛІМ.....	12
2.1 Пәндік аймақты талдау және есептің қойылымы.....	12
2.2 Қолданыстағы шешімдерге (аналогтарға) шолу.....	12
2.3 Жүйе архитектурасы.....	13
2.4 Эскиздік жобалау: пайдалану нұсқалары.....	14
2.5 Деректер қорын жобалау.....	15
2.6 Серверлік бөлікті іске асыру.....	16
2.7 Клиенттік бөлікті іске асыру.....	17
2.8 Электронды бекіту және қолтаңба модулі.....	18
2.9 Хабарламалар, Telegram бот, AI-көмекші және рейтингтер.....	19
2.10 Тестілеу.....	20
2.11 Жүйенің пайдаланушы интерфейсі (скриншоттар).....	21
2.12 Орнату және пайдаланушы нұсқаулығы.....	25
2.13 Жүйенің қауіпсіздігі.....	25
2.14 Жобаны жетілдіру перспективалары.....	26
3 ЭКОНОМИКАЛЫҚ БӨЛІМ.....	27
4 ЕҢБЕКТІ ҚОРҒАУ ЖӘНЕ ҚАУІПСІЗДІК.....	36
4.1 Жұмыс орнын ұйымдастыру және эргономика.....	36
4.2 Жарықтандыру және көру қабілетіне түсетін жүктеме.....	36
4.3 Электр қауіпсіздігі.....	37
4.4 Жұмыс орнындағы микроклимат.....	37
4.5 Ақпараттық қауіпсіздік.....	37
4.6 Психологиялық аспектілер мен өрт қауіпсіздігі.....	38
4.7 Қорытынды.....	38
ҚОРЫТЫНДЫ.....	39
ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР ТІЗІМІ.....	40
Қосымша А. Деректер қорының схемасы (D1, SQLite).....	41
Қосымша Б. Аутентификация модулі (JWT).....	44
Қосымша В. Сервердің кіру нүктесі (Нопс).....	47
Қосымша Г. Электронды қолтаңба модулі.....	52
Қосымша Д. Клиенттік API қабаты.....	54
Қосымша Е. Деректерді валидациялау схемалары (Zod).....	58

Қосымша Ж. Тәрбие сағаттарының бизнес-логикасы.....	62
Қосымша З. Сабақты бекіту және құжатпен жұмыс модулі.....	69
Қосымша И. Суреттер мен скриншоттарды орналастыру картасы.....	75

КІРІСПЕ

Қазіргі білім беру жүйесінде оқу процесін цифрландыру басым бағыттардың біріне айналды. Мектептер мен колледждерде тек сабақтарды ғана емес, сонымен қатар тәрбие жұмысын да жүйелі түрде жоспарлау, бақылау және есепке алу қажеттілігі артып отыр. Тәрбие сағаты (тәрбие сағаты) – оқушылар мен студенттердің бойында азаматтық, рухани-адамгершілік және әлеуметтік құндылықтарды қалыптастыруға бағытталған оқу-тәрбие процесінің ажырамас бөлігі. Алайда көптеген оқу орындарында тәрбие сағатын жоспарлау, өткізу фактісін растау және ай сайынғы есепті жинау әлі күнге дейін қағаз журналдар мен бытыраңқы құжаттар арқылы жүргізіледі.

Дәстүрлі қағаз негізіндегі тәсілдің бірқатар кемшіліктері бар. Біріншіден, құжаттар жоғалуы немесе кешігіп тапсырылуы мүмкін. Екіншіден, кураторлардың есептерін қолмен жинау әкімшілік қызметкерлердің едәуір уақытын алады. Үшіншіден, тәрбие сағатының нақты өткізілгенін растайтын бірыңғай дерек көзі болмайды, сондықтан мониторинг қиындайды. Төртіншіден, оқушылар мен ата-аналарды алдағы іс-шаралар туралы дер кезінде хабардар ету жүйеленбеген.

Осы мәселелерді шешу үшін осы дипломдық жобада «Тәрбие Сағаты» атты веб-негіздегі ақпараттық жүйе әзірленді. Жүйе тәрбие сағатын электронды түрде жоспарлауға, оны өткізу нәтижесін бекітуге, қатысуды есепке алуға, Word форматындағы құжаттарды электронды қолтаңбамен бекітуге, Telegram арқылы хабарламалар жіберуге және ай сайынғы есептерді автоматты түрде қалыптастыруға мүмкіндік береді. Жүйе үш негізгі пайдаланушы рөліне – әкімшіге, кураторға (мұғалімге) және студентке – арналған.

Жобаның өзектілігі білім беру саласындағы құжат айналымын цифрландыру қажеттілігімен айқындалады. Бұлтты технологиялар мен заманауи веб-фреймворктер бүгінде күрделі инфрақұрылым құрмай-ақ, сенімді әрі масштабталатын жүйелер жасауға мүмкіндік береді. Жоба Cloudflare Workers платформасында іске асырылды, бұл серверді бөлек жалдаудың қажетін жояды және қосымшаны пайдаланушыға жақын «шеткі» (edge) түйіндерде орындауды қамтамасыз етеді.

Жобаның мақсаты – тәрбие сағатын жоспарлау, бекіту, қатысуды есепке алу және есеп беру процестерін автоматтандыратын, рөлдік қол жеткізуі бар, заманауи технологиялық стекте құрылған ақпараттық жүйені жобалау және іске асыру.

Қойылған мақсатқа жету үшін келесі міндеттер шешілді:

- тәрбие сағатын басқарудың пәндік аймағын зерттеу және қолданыстағы шешімдерге шолу жасау;
- жүйеге қойылатын функционалдық және функционалдық емес талаптарды анықтау;

- клиент–серверлік архитектураны және деректер қорының құрылымын жобалау;
- Cloudflare Workers, Hono және D1 негізінде REST API серверін әзірлеу;
- React және Tailwind CSS негізінде бір беттік веб-қосымша (SPA) құру;
- электронды қолтаңба мен Word-құжатты бекіту модулін іске асыру;
- Telegram бот пен кезекке негізделген хабарлама жүйесін енгізу;
- жүйені тестілеу, экономикалық есеп пен еңбек қауіпсіздігі бөлімдерін дайындау.

Зерттеу нысаны – оқу орнындағы тәрбие жұмысын жоспарлау және есепке алу процесі. Зерттеу пәні – осы процесті автоматтандыратын веб-негіздегі ақпараттық жүйені жобалау және әзірлеу әдістері мен құралдары.

Жобаның практикалық маңызы – әзірленген жүйе нақты оқу орындарында тәрбие сағатының есебін жүргізуге, кураторлар мен әкімшіліктің уақытын үнемдеуге және құжат айналымын ашық әрі бақыланатын етуге мүмкіндік береді. Түсіндірме жазба кіріспеден, төрт негізгі бөлімнен, қорытындыдан, пайдаланылған әдебиеттер тізімінен және қосымшалардан тұрады.

1 ЖАЛПЫ БӨЛІМ

1.1 Бағдарламалық Өнімнің сипаттамасы

«Тәрбие Сағаты» – оқу орнындағы тәрбие сағаттарын электронды түрде жоспарлауға, бекітуге және есепке алуға арналған веб-қосымша. Жүйенің негізгі мақсаты – қағаз журналдар мен бытыраңқы құжаттарды біртұтас цифрлық платформамен алмастыру, тәрбие сағатының өткізілу фактісін растау процесін айқын әрі бақыланатын ету.

Жүйе браузер арқылы ашылады, сондықтан пайдаланушы жағында арнайы бағдарлама орнатудың қажеті жоқ. Қосымша компьютерде де, смартфонда да дұрыс көрсетіледі, себебі интерфейс бейімделгіш (responsive) тәсілмен жобаланған. Интерфейс екі тілде – қазақ және орыс тілдерінде – қолжетімді.

Жүйе үш негізгі рөлге қызмет көрсетеді. Әкімші (admin) оқу орнының пайдаланушылары мен сыныптарын басқарады, тәрбие сағаттарын бекітеді немесе қайтарады, есептерді көреді. Куратор (teacher) өзі жетекшілік ететін топ үшін тәрбие сағаттарын жоспарлайды, аудитория мен уақытты белгілейді, өткізілген сабақты аяқталды деп белгілейді, қатысуды тіркейді және құжаттарды бекітуге жібереді. Студент (student) өзіне қатысты тәрбие сағаттарын, бағаларын және хабарламаларды көреді, сонымен қатар өткізілген сабаққа баға (рейтинг) бере алады.

Бағдарламалық өнім ретінде жүйенің бірнеше қырын бөліп көрсетуге болады. Біріншісі – басқару құралы: тәрбие сағаттарының тізімі, мәртебелер, күнтізбе және аудиторияларды тағайындау. Екіншісі – құжатпен жұмыс істеу қабаты: Word-құжатты жүктеу, оны бекітуге жіберу, электронды қолтаңбаны құжатқа ендіру. Үшіншісі – коммуникация мен аналитика: Telegram арқылы хабарлама, AI-көмекші, рейтингтер және ай сайынғы есептер.

Жүйенің маңызды ерекшелігі – оның көп мекемелік (multi-tenant) болуы. Әрбір оқу орны (мектеп немесе колледж) жеке дерек кеңістігінде жұмыс істейді: бір мекеменің пайдаланушылары екінші мекеменің деректерін көре алмайды. Бұл деректердің оқшаулануын және құпиялылығын қамтамасыз етеді.

1.2 Пәндік аймақты талдау және қолданыстағы шешімдерге шолу

Тәрбие сағатын басқару процесі бірнеше кезеңнен тұрады: жоспарлау, өткізу, өткізу фактісін растау, қатысуды тіркеу және есеп беру. Дәстүрлі тәсілде бұл кезеңдер қағаз журналдар, кесте файлдары және жеке хабарласулар арқылы жүзеге асырылады. Мұндай тәсіл деректердің тұтастығын қамтамасыз етпейді және адам факторынан туындайтын қателіктерге бейім.

Қолданыстағы аналогтарды талдау барысында бірнеше санаттағы шешімдер қарастырылды. Бірінші санат – жалпы мақсаттағы электронды

журнал жүйелері (мысалы, Күнделік, BilimLand сияқты платформалар). Олар сабақтар мен бағаларды есепке алуға бағытталған, бірақ тәрбие жұмысын, оның ішінде тәрбие сағатын бекіту мен құжаттарды электронды қолтаңбамен растау процесін толық қамтымайды.

Екінші санат – құжат айналымының жалпы жүйелері (СЭД). Олар қуатты болғанымен, оқу орнының нақты қажеттіліктеріне бейімделмеген, лицензиялары қымбат және енгізу күрделі. Үшінші санат – Google Workspace немесе Microsoft 365 сияқты кеңсе құралдары жиынтығы. Олар икемді болғанымен, тәрбие сағатының өзіндік логикасын (мәртебелер, бекіту тізбегі, рөлдік құқықтар) қамтымайды және процесті автоматтандырмайды.

Жүргізілген талдау қолданыстағы шешімдердің тәрбие сағатын басқару міндетіне толық сәйкес келмейтінін көрсетті. Сондықтан осы пәндік аймаққа арнайы бейімделген, жеңіл әрі заманауи технологиялық стекте құрылған меншікті жүйені әзірлеу негізделген. Осы жүйенің артықшылықтары 1.1-кестеде қолданыстағы тәсілдермен салыстырмалы түрде келтірілген.

1.1-кесте – Тәрбие сағатын басқару тәсілдерін салыстыру

Критерий	Қағаз журнал	Жалпы СЭД	«Тәрбие Сағаты»
Тәрбие сағатының логикасы	Жоқ	Жартылай	Толық
Электронды қолтаңбамен бекіту	Жоқ	Бар	Бар
Рөлдік қол жеткізу	Жоқ	Бар	Бар
Telegram хабарламалары	Жоқ	Сирек	Бар
Ай сайынғы автоесеп	Қолмен	Жартылай	Автоматты
Енгізу құны	Төмен	Жоғары	Төмен

Кестеден көріп отырғандай, әзірленген жүйе тәрбие сағатын басқарудың арнайы логикасын толық қамтиды әрі енгізу құны жағынан тиімді болып табылады. Осылайша меншікті шешімді әзірлеу пәндік аймақтың талаптарына барынша сәйкес келеді.

1.3 Қолданылған технологиялар мен тілдер

Жобаның технологиялық негізі заманауи, бірақ артық ауыр емес құралдардан таңдалды. Басты мақсат – күрделі серверлік инфрақұрылым құрмай-ақ, сенімді әрі масштабталатын веб-жүйе жасау. Барлық бөліктер бірыңғай тіл – TypeScript тілінде жазылды, бұл клиент пен сервер арасында типтердің сәйкестігін қамтамасыз етеді және қателерді әзірлеу кезеңінде анықтауға көмектеседі.

Негізгі бағдарламалау тілі ретінде TypeScript таңдалды. TypeScript – JavaScript тіліне статикалық типтендіруді қосатын тіл. Ол үлкен жобаларда кодтың сенімділігін арттырады, өңдеу барысында қателерді ерте байқауға мүмкіндік береді және код базасын қолдауды жеңілдетеді. Жобада қатаң

режим (strict mode) қосылған, бұл айнымалылардың түрлерін мұқият бақылауды талап етеді.

Серверлік бөлік Cloudflare Workers платформасында орындалады. Cloudflare Workers – код «шеткі» (edge) серверлерде, яғни пайдаланушыға географиялық жақын түйіндерде орындалатын serverless есептеу ортасы. Бұл кідірісті азайтады, автоматты масштабтауды қамтамасыз етеді және серверді бөлек басқару қажеттілігін жояды. API маршрутизациясы үшін жеңіл әрі жылдам Hono веб-фреймворкі қолданылды.

Деректерді сақтау үшін Cloudflare D1 қолданылды. D1 – SQLite қозғалтқышына негізделген, Workers-пен тығыз интеграцияланған реляциялық деректер қоры. Ол SQL сұраныстарын, шетелдік кілттерді (foreign keys) және индекстерді қолдайды. Қосымша деректер (мысалы, екілік файлдар мен жеделдеткіш) үшін Cloudflare KV кілт-мән қоймасы пайдаланылды.

Асинхронды тапсырмалар, атап айтқанда хабарламаларды жіберу, Cloudflare Queues кезек қызметі арқылы орындалады. Кезек хабарлама жіберуді негізгі сұраныстан бөліп алуға және сәтсіз жіберулерді қайта орындауға (retry) мүмкіндік береді.

Клиенттік бөлік React 18 кітапханасында жазылды. Жобаны жинау үшін Vite құралы, стильдеу үшін Tailwind CSS, күй-жайды (state) басқару үшін Zustand, ал графиктер мен диаграммаларды салу үшін Recharts кітапханасы қолданылды. Аутентификация JWT (JSON Web Token) технологиясы негізінде, Workers-тің кірістірілген криптографиялық API-і (Web Crypto) арқылы сыртқы кітапханаларсыз жүзеге асырылды. Деректерді валидациялау үшін Zod схемалар кітапханасы қолданылды, ол клиент пен серверде ортақ пайдаланылады.

Кураторларға арналған AI-көмекші Google Gemini моделі негізінде жұмыс істейді. Жобада қолданылған негізгі технологиялардың тізімі 1.2-кестеде келтірілген.

1.2-кесте – Жобада қолданылған негізгі технологиялар

Қабат	Технология	Тағайындалуы
Тіл	TypeScript	Барлық модульдер үшін бірыңғай типтендірілген тіл
Сервер	Cloudflare Workers, Hono	REST API және бизнес-логика
Деректер қоры	Cloudflare D1 (SQLite)	Реляциялық деректерді сақтау
Қойма	Cloudflare KV	Файлдар, жеделдеткіш, лимиттер
Кезек	Cloudflare Queues	Хабарламаларды асинхронды жіберу
Клиент	React 18, Vite, Tailwind	Веб-интерфейс (SPA)
Күй	Zustand	Клиенттік күй-жайды басқару
Валидация	Zod	Кіріс деректерді тексеру
Аутентификация	JWT (Web Crypto)	Қол жеткізуді бақылау
AI	Google Gemini	Кураторға арналған

		көмекші
--	--	---------

Таңдалған технологиялық стек жобаның жеңіл, сенімді және болашақта кеңейтуге ыңғайлы болуын қамтамасыз етеді. Барлық құрауыштардың бірыңғай экожүйеде (Cloudflare) орналасуы әзірлеу мен орналастыруды жеңілдетеді.

1.4 Жүйеге қойылатын талаптар

Жүйені жобалаудың алдында оған қойылатын талаптар нақтыланды. Талаптар екі топқа бөлінеді: функционалдық (жүйе нені орындауы керек) және функционалдық емес (жүйе қандай сапа көрсеткіштеріне сай болуы керек).

Функционалдық талаптар жүйенің нақты мүмкіндіктерін сипаттайды:

- телефон нөмірі мен біржолғы код (ОТР) арқылы аутентификация;
- рөлге байланысты қол жеткізуді бөлу (әкімші, куратор, студент);
- тәрбие сағатын құру, өңдеу, тағайындау және жою;
- аудитория мен уақыт аралығын (пара/слот) белгілеу;
- тәрбие сағатының мәртебесін басқару (жоспарланған, аяқталған, бас тартылған, ауыстырылған);
- қатысуды тіркеу (қатысты, қатыспады, кешікті, дәлелді себеп);
- Word-құжатты жүктеу, бекітуге жіберу және электронды қолтаңба ендіру;
- Telegram арқылы хабарлама жіберу және ай сайынғы есеп қалыптастыру;
- студенттердің бағасы (рейтинг) мен AI-көмекші арқылы кеңес алу.

Функционалдық емес талаптар жүйенің сапасына қойылады:

- қауіпсіздік: барлық жабық маршруттар JWT-токенмен қорғалуы тиіс;
- деректердің оқшаулануы: бір мекеме екінші мекеменің деректерін көрмеуі тиіс;
- өнімділік: жиі сұраныстар индекстелген кестелер арқылы тез орындалуы тиіс;
- сенімділік: хабарламалар кезек арқылы кепілді жеткізілуі тиіс;
- қолжетімділік: интерфейс екі тілде және мобильді құрылғыларда жұмыс істеуі тиіс;
- кеңейтілгіштік: жаңа модульдер бар архитектураны бұзбай қосылуы тиіс.

Пайдаланушы рөлдері мен олардың құқықтары 1.3-кестеде жинақталған. Бұл кесте жүйедегі рөлдік модельдің негізін құрайды.

1.3-кесте – Пайдаланушы рөлдері және құқықтары

Рөл	Негізгі құқықтары
Әкімші (admin)	Пайдаланушылар мен сыныптарды басқару, тәрбие сағатын бекіту/қайтару, есептер, баптаулар
Куратор (teacher)	Тәрбие сағатын жоспарлау, өткізу,

	қатысуды тіркеу, құжатты бекітуге жіберу
Студент (student)	Өз сабақтарын, бағаларын және хабарламаларды көру, рейтинг беру

Анықталған талаптар жүйенің архитектурасы мен деректер қорын жобалаудың негізі болды. Олардың орындалуы екінші, арнайы бөлімде толық сипатталады.

2 АРНАЙЫ БӨЛІМ

2.1 Пәндік аймақты талдау және есептің қойылымы

Автоматтандыру нысаны – оқу орнындағы тәрбие жұмысын жоспарлау, өткізу және есепке алу процесі. Қазіргі тәжірибеде куратор (топ жетекшісі) ай сайын тәрбие сағаттарын жоспарлайды, оларды өткізеді, қатысуды тіркейді және нәтижесі бойынша есеп дайындайды. Бұл есеп әкімшілікке (оқу ісі жөніндегі орынбасарға) тапсырылып, тексеріледі және бекітіледі.

Пәндік аймақты талдау көрсеткендей, процестің негізгі қатысушылары үш рөлге бөлінеді: әкімші (мекеме деңгейіндегі басқару және бекіту), куратор (сабақты жоспарлау, өткізу, есеп беру) және студент (сабақ туралы ақпаратты көру және баға беру). Әр рөлдің өз әрекеттер жиыны мен қол жеткізу деңгейі болады.

Талдау нәтижесінде жүйеге қойылатын негізгі функционалдық талаптар анықталды:

- телефон нөмірі мен біржолғы код (ОТР) арқылы қауіпсіз аутентификация;
- рөлге негізделген қол жеткізуді басқару (әкімші, куратор, студент);
- тәрбие сағаттарын құру, өңдеу, өткізу және есепке алу;
- қатысуды тіркеу және ай сайынғы есепті автоматты қалыптастыру;
- Word-құжатты электронды қолтаңбамен бекіту;
- Telegram арқылы хабарлама жіберу және ботпен өзара әрекеттесу;
- студенттердің сабаққа баға беруі (рейтинг) және оны сүзгілеу.

Функционалдық емес талаптарға қауіпсіздік (деректерді мекемелер бойынша оқшаулау, JWT-аутентификация), масштабталу (бұлтты «шеткі» инфрақұрылым), қолжетімділік (браузер арқылы, орнатусыз) және интерфейстің екі тілде (қазақ, орыс) болуы жатады. Осы талаптар жобаның архитектурасы мен технологиялық стегін таңдауды айқындады.

2.2 Қолданыстағы шешімдерге (аналогтарға) шолу

Есепті шешу алдында ұқсас міндеттерді орындайтын қолданыстағы шешімдерге талдау жасалды. Білім беру саласында электронды журнал және құжат айналымы жүйелері кеңінен қолданылады, алайда олардың көпшілігі тәрбие сағатын электронды бекіту мен қолтаңба ендіру міндетін толық қамтымайды. Негізгі аналогтардың салыстырмалы талдауы 2.1-кестеде келтірілген.

2.1-кесте – Аналогтарды салыстырмалы талдау

Шешім	Артықшылығы	Кемшілігі
Қағаз журнал	Қарапайым, жабдық қажет емес	Жоғалуы мүмкін, бақылау қиын, есеп қолмен жиналады
Әмбебап электронды журнал	Сабақ кестесі мен бағаны есепке алады	Тәрбие сағатын бекіту мен электронды қолтаңба жоқ

Мессенджер-чаттар	Жедел хабарласу	Құрылымдалмаған, есеп пен мұрағат жоқ
«Тәрбие Сағаты» (осы жоба)	Бекіту, қолтаңба, есеп, Telegram, рейтинг	Жаңа жоба, бейімдеу қажет

Талдау көрсеткендей, қолданыстағы шешімдер тәрбие сағатын электронды бекіту, құжатқа қолтаңба ендіру және рөлдік есеп беру міндеттерін бір жерде шешпейді. Сондықтан осы дипломдық жобада өзіндік шешім әзірлеу қажеттілігі негізделді.

2.3 Жүйе архитектурасы

Жүйе клиент–серверлік архитектура бойынша құрылған. Бұл тәсілде көрсету логикасы (клиент) мен деректерді өңдеу логикасы (сервер) бір-бірінен бөлінген, олардың арасындағы байланыс HTTP протоколы арқылы JSON форматында жүзеге асырылады. Мұндай бөліну әрбір бөлікті бөлек әзірлеуге, тестілеуге және масштабтауға мүмкіндік береді.

Архитектура үш негізгі құрауыштан тұрады. Бірінші құрауыш – клиенттік қосымша (frontend): браузерде орындалатын React SPA. Ол пайдаланушы интерфейсін көрсетеді және серверге API сұраныстарын жібереді. Екінші құрауыш – API сервері (worker): Node фреймворкінде жазылған, Cloudflare Workers-та орындалатын қосымша. Ол сұраныстарды қабылдайды, бизнес-логиканы орындайды және деректер қорымен жұмыс істейді. Үшінші құрауыш – бот және кезек өңдеушісі (bot-worker): Telegram хабарламаларын өңдейтін және кезектен келген хабарламаларды жіберетін жеке Worker.

Деректерді сақтау деңгейінде үш қызмет қолданылады: D1 (реляциялық деректер), KV (екілік файлдар мен жеделдеткіш) және Queues (асинхронды хабарламалар кезегі). Барлық құрауыштар Cloudflare экожүйесінде орналасқандықтан, олардың арасындағы байланыс жылдам әрі қауіпсіз жүреді. Жүйенің жалпы архитектурасы 2.1-суретте көрсетілген.



2.1-сурет – Жүйенің жалпы архитектуралық схемасы

Сұраныстың өңделу жолы келесідей. Пайдаланушы браузерде әрекет жасайды → React қосымшасы API серверіне HTTP сұранысын жібереді → сервердегі аралық бағдарламалар (middleware) сұранысты тексереді (CORS, дене көлемінің шегі, JWT-аутентификация) → сәйкес маршрут өңдеушісі бизнес-логиканы орындайды және D1-ден деректерді алады → нәтиже JSON түрінде клиентке қайтарылады. Хабарлама жіберу қажет болған жағдайда сервер тапсырманы кезекке қояды, ал bot-worker оны фонда өңдейді.

Сервердегі аралық бағдарламалардың (middleware) тізбегі 2.2-кестеде келтірілген. Олар әрбір сұранысқа дәйекті түрде қолданылады.

2.2-кесте – Сервердің аралық бағдарламалары (middleware)

Middleware	Қызметі
corsMiddleware	Браузераралық сұраныстарды (CORS) реттеу
bodyLimit	Сұраныс денесінің көлемін шектеу (DoS-тан қорғау)
authMiddleware	JWT-токенді тексеру және пайдаланушыны анықтау
requireRole	Рөлге байланысты қол жеткізуді шектеу
rateLimit	Сұраныс жиілігін шектеу (KV арқылы)

Жоғарыда сипатталған архитектура жауапкершіліктің анық бөлінуін қамтамасыз етеді және жүйені сенімді әрі масштабталатын етеді.

2.4 Эскиздік жобалау: пайдалану нұсқалары

Жүйенің функционалдық моделін көрнекі түрде ұсыну үшін UML белгілеуіндегі пайдалану нұсқалары (use-case) диаграммасы құрылды. Ол актёрлер (Әкімші, куратор, студент) мен олардың жүйемен орындайтын негізгі әрекеттерінің арасындағы байланысты көрсетеді.

Әкімші пайдаланушылар мен сыныптарды басқарады, құжаттарды бекітеді және есептерді көреді. Куратор тәрбие сағаттарын құрады, қатысуды тіркейді, құжаттарды бекітуге жібереді және AI-көмекшіні пайдаланады. Студент сабақ туралы ақпаратты көреді және өткізілген сабаққа баға береді. Барлық актёрлер үшін ортақ әрекет – жүйеге кіру (аутентификация). Use-case диаграммасы 2.2-суретте келтірілген.

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]

Қайдан түсіру: draw.io / PlantUML-да use-case диаграммасын салып, суретін осы жерге қойыңыз

2.2-сурет – Пайдалану нұсқаларының (use-case) диаграммасы

2.5 Деректер қорын жобалау

Деректер қоры жүйенің негізі болып табылады. Ол Cloudflare D1 (SQLite) қозғалтқышында орналасқан және реляциялық модель бойынша жобаланған. Кестелер арасындағы байланыстар шетелдік кілттермен (foreign keys), ал деректердің тұтастығы ЧЕСК-шектеулерімен қамтамасыз етіледі. Жиі сұралатын бағандарға индекстер құрылды, бұл сұраныстардың жылдамдығын арттырады.

Схеманың орталық кестесі – schools (мекемелер). Әрбір пайдаланушы, сынып және тәрбие сағаты белгілі бір мекемеге тиесілі. Бұл көп мекемелік (multi-tenant) модельдің негізін құрайды: school_id өрісі арқылы деректер оқшауланады. users кестесінде пайдаланушылар, олардың рөлі, телефоны және Telegram-идентификаторы сақталады. classes кестесі сыныптарды (топтарды) кураторға және оқу жылына байланыстырады, ал class_students кестесі студенттерді сыныпқа бекітеді.

Жүйенің басты нысаны – tarbie_sessions кестесі. Онда тәрбие сағатының тақырыбы, жоспарланған және нақты күні, мәртебесі, ұзақтығы, аудиториясы мен уақыт аралығы сақталады. session_attendance кестесінде әр сабақтағы студенттердің қатысуы тіркеледі. notifications_log кестесінде жіберілген хабарламалардың журналы, ал notification_templates кестесінде хабарлама мәтіндерінің үлгілері сақталады. Деректер қорының негізгі кестелері 2.3-кестеде сипатталған.

2.3-кесте – Деректер қорының негізгі кестелері

Кесте	Тағайындалуы
schools	Оқу орындары (мекемелер)
users	Пайдаланушылар: әкімші, куратор, студент
classes	Сыныптар (топтар) және оқу жылы
class_students	Студенттің сыныпқа тиесілігі
tarbie_sessions	Тәрбие сағаттары (негізгі нысан)

session_attendance	Қатысуды тіркеу
notifications_log	Жіберілген хабарламалар журналы
notification_templates	Хабарлама мәтіндерінің үлгілері
user_signatures	Пайдаланушылардың электронды қолтаңбалары
session_ratings	Студенттердің сабаққа берген бағасы

Кестелер арасындағы байланыстар мен өрістерді көрсететін ER-диаграмма 2.3-суретте келтірілген. Барлық байланыстар каскадтық жою (ON DELETE CASCADE) немесе шектеу (RESTRICT) ережелерімен реттелген, бұл деректердің тұтастығын сақтайды.



2.3-сурет – Деректер қорының ER-диаграммасы

2.6 Серверлік бөлікті іске асыру

Серверлік бөлік Hono фреймворкінде жазылған. Hono – Cloudflare Workers үшін оңтайландырылған жеңіл әрі жылдам веб-фреймворк. Бағдарламаның кіру нүктесінде (worker.ts) барлық маршрут топтары тіркеледі: аутентификация, тәрбие сағаттары, қатысу, хабарламалар, әкімшілік, есептер, бағалар, оқиғалар, ашық сабақтар, қолдау қызметі, Telegram бот, рейтингтер, курстар, AI-көмекші, премиум, кураторларды басқару, қолтаңбалар және сабақты бекіту.

Аутентификация телефон нөмірі мен біржолғы код (OTP) арқылы жүзеге асырылады. Пайдаланушы нөмірін енгізгенде сервер код жасап, оны Telegram арқылы жібереді. Кодты растағаннан кейін сервер JWT-токен қайтарады. JWT (JSON Web Token) – пайдаланушының жеке басын растайтын, цифрлық қолтаңбамен қорғалған токен. Жобада ол HMAC-SHA256 алгоритмімен, Workers-тің кірістірілген Web Crypto API-і арқылы, сыртқы кітапханаларсыз қол қойылады және тексеріледі.

Әрбір жабық маршрут authMiddleware аралық бағдарламасынан өтеді. Ол Authorization тақырыбындағы токенді тексереді, оның жарамдылығы мен

мерзімін бақылайды. Токен жарамды болса, пайдаланушының идентификаторы, рөлі және мекемесі сұраныс контекстіне жазылады. requireRole аралық бағдарламасы нақты рөлдерге ғана рұқсат береді (мысалы, тек әкімшіге).

Барлық API жауаптары бірыңғай форматта қайтарылады. Сәтті жауап { success: true, data: ... } құрылымына, ал қателер { success: false, code, message } құрылымына ие. Бұл клиенттік бөлікте қателерді біркелкі өңдеуге мүмкіндік береді. Деректерді өзгертетін сұраныстар Zod схемалары арқылы тексеріледі, бұл жарамсыз деректердің деректер қорына түсуін болдырмайды.

API маршруттарының негізгі тобы 2.4-кестеде келтірілген. Барлығы 18-ден астам маршрут тобы және 70-тен астам жеке эндпойнт бар.

2.4-кесте – Негізгі API эндпойнттері

Әдіс	Жол	Сипаттамасы
POST	/api/auth/login	Телефон бойынша OTP сұрау
POST	/api/auth/verify	OTP растау, JWT алу
GET	/api/auth/me	Ағымдағы пайдаланушы
GET	/api/sessions	Тәрбие сағаттарының тізімі
POST	/api/sessions	Тәрбие сағатын құру
PATCH	/api/sessions/:id/complete	Сабақты аяқталды деп белгілеу
POST	/api/attendance/:id	Қатысуды тіркеу
POST	/api/lesson-approvals	Құжатты бекітуге жіберу
POST	/api/signatures/me	Электронды қолтаңбаны сақтау
GET	/api/reports/monthly	Айлық есеп
POST	/api/assistant	AI-көмекшіге сұраныс

2.7 Клиенттік бөлікті іске асыру

Клиенттік бөлік – React 18 кітапханасында жазылған бір беттік қосымша (SPA). Бір беттік қосымшада бет толық қайта жүктелмейді: пайдаланушы әрекет жасағанда тек қажетті бөлік жаңартылады, бұл жұмысты жылдам әрі үздіксіз етеді. Жоба Vite құралымен жиналады, ол әзірлеу кезінде кодты лезде жаңартуды (HMR) қамтамасыз етеді.

Қосымшаның құрылымы беттер (pages) мен компоненттерге (components) бөлінген. Әрбір бет белгілі бір функцияға жауап береді: кіру беті (LoginPage), басты панель (DashboardPage), тәрбие сағаттары (SessionsPage), бағалар (GradesPage), есептер (ReportsPage), сабақты бекіту (LessonApprovalsPage), баптаулар (SettingsPage), AI-көмекші (AssistantPage) және басқалар. Қайталанатын элементтер (макет, аватар, мәртебе белгісі, қолтаңба өрісі) жеке компоненттерге бөлініп шығарылды.

Қолданбаның жаһандық күй-жайы (ағымдағы пайдаланушы, токен, тіл, тақырып) Zustand кітапханасы арқылы басқарылады. Zustand – React үшін жеңіл әрі қарапайым күй-жай менеджері. Сервермен байланыс api.ts

модуліндегі бірыңғай ApiClient клиенті арқылы жүреді. Бұл клиент токенді автоматты түрде сұраныстарға қосады, жауаптарды талдайды, қателерді өңдейді және 401 (авторизацияланбаған) жауап келгенде пайдаланушыны кіру бетіне қайтарады.

Стильдеу Tailwind CSS утилиталық фреймворкі арқылы орындалды. Tailwind дайын кластарды қолдану арқылы стильді тікелей разметкада жазуға мүмкіндік береді, бұл әзірлеуді жылдамдатады және интерфейстің біртұтастығын сақтайды. Интерфейс жарық және қараңғы тақырыпты (theme), сондай-ақ қазақ және орыс тілдерін қолдайды. Дайын интерфейс экрандарының скриншоттары 2.11-тармақта келтірілген.

2.8 Электронды бекіту және Қолтаңба модулі

Жобаның ерекше әрі негізгі модульдерінің бірі – тәрбие сағатының құжатын электронды түрде бекіту және оған қолтаңба ендіру тетігі. Бұл модуль қағаз құжатқа қойылатын қол мен мөрді цифрлық баламамен алмастырады.

Процесс келесідей жүреді. Куратор тәрбие сағатын өткізгеннен кейін оның Word форматындағы (.docx) есеп құжатын жүйеге жүктейді және оны бекітуге жібереді. Әкімші бұл құжатты қарап шығып, оны бекітеді немесе түсініктемемен қайтарады. Бекіту кезінде жүйе құжатқа кураторнің және Әкімшінің электронды қолтаңбаларын автоматты түрде ендіреді.

Электронды қолтаңбаның өзі пайдаланушы тарапынан алдын ала салынады. Жүйеде қолтаңба алаңы (SignaturePad) бар: пайдаланушы тінтуірмен немесе сенсорлық экранмен қолын салады, ол сурет (base64 PNG) түрінде user_signatures кестесінде сақталады. Студенттерге қолтаңба қажет емес, ал кураторлар мен Әкімшілерге міндетті. Бірінші кіргенде жүйе қолтаңбаны салуды ұсынады.

Қолтаңбаны .docx құжатына ендіру – техникалық тұрғыдан күрделі тапсырма. .docx файлы шын мәнінде ішінде XML файлдары бар ZIP мұрағаты. Қолтаңбаны ендіру үшін клиенттік бөлікте JSZip кітапханасы қолданылады: ол құжатты ашады, ішіндегі XML-ге қолтаңба суретін сурет ретінде кірістіреді және мұрағатты қайта жинайды. Осылайша бекітілген құжатта қолтаңбалар тиісті орындарда көрінеді. Бекіту процесінің алгоритмі 2.4-суретте келтірілген.

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]

Қайдан түсіру: блок-схеманы draw.io-да салыңыз (жоғарыдағы қадамдар бойынша)

2.4-сурет – Тәрбие сағатын бекіту процесінің алгоритмі

Сабақтың мәртебесі бекіту процесінде өзгеріп отырады. Мәртебелер мен олардың мағынасы 2.5-кестеде келтірілген.

2.5-кесте – Тәрбие сағатының мәртебелері

Мәртебе	Мағынасы
pending_approval	Бекітуді күтуде
planned	Жоспарланған
completed	Аяқталған
cancelled	Болдырылмаған
rescheduled	Басқа күнге ауыстырылған

2.9 Хабарламалар, Telegram бот, AI-көмекші және рейтингтер

Хабарлама жүйесі кезекке (Cloudflare Queues) негізделген. Хабарлама жіберу қажет болғанда сервер тапсырманы кезекке қояды да, сұранысты бірден аяқтайды. Кезекті жеке Worker (bot-worker) өңдейді: ол хабарламаны Telegram арқылы жібереді. Егер жіберу сәтсіз болса, жүйе экспоненциалды кідіріспен үш ретке дейін қайталайды, ал сәтсіздік журналға (notifications_log) жазылады. Бұл хабарламалардың кепілді жеткізілуін қамтамасыз етеді.

Хабарламалар бірнеше оқиғаға байланысты жіберіледі: тәрбие сағаты жоспарланғанда, еске салу ретінде (сабаққа бір күн қалғанда), сабақ аяқталғанда, күн ауысқанда немесе студент сабаққа қатыспағанда. Еске салулар күн сайын белгіленген уақытта автоматты түрде (cron) іске қосылады. Хабарлама мәтіндері үлгілер арқылы екі тілде қалыптасады.

Telegram бот пайдаланушымен өзара әрекеттесудің қосымша арнасы болып табылады. Ол /start (аккаунтты байланыстыру), /my_sessions (алдағы тәрбие сағаттары) және /confirm (сабақты растау) командаларын өңдейді. Бот арқылы пайдаланушы веб-интерфейске кірмей-ақ негізгі әрекеттерді орындай алады. Telegram-ды веб-аккаунтпен байланыстыру UUID негізіндегі біржолғы «сиқырлы сілтеме» (magic token) арқылы жүзеге асырылады.

Кураторларға арналған AI-көмекші Google Gemini тілдік моделінде жұмыс істейді. Көмекші тек тәрбие жұмысына қатысты сұрақтарға жауап беруге қатаң шектелген: жүйелік нұсқаулық (prompt) арқылы ол бағдарламалау, саясат немесе бөгде тақырыптарға жауап беруден бас тартады. Артық пайдалануды болдырмау үшін тәуліктік лимит (30 сұраныс) енгізілген, ол KV қоймасы арқылы есептеледі.

Рейтинг модулі студенттерге өткізілген тәрбие сағатына баға қоюға мүмкіндік береді. Жүйеде «ақылды сүзгі» (smart filter) бар: ол күдікті немесе мағынасыз пікірлерді автоматты түрде жарамсыз деп белгілейді. Мысалы, өте жоғары немесе төмен баға себепсіз қойылса, пікір тым қысқа болса немесе бір әріптің қайталануынан тұрса, ол есепке алынбайды. Бұл рейтингтің объективтілігін арттырады. Сүзгі ережелері 2.6-кестеде келтірілген.

2.6-кесте – Рейтинг сүзгісінің (smart filter) ережелері

Жағдай	Нәтиже
Шектік баға (≤ 2 не ≥ 10) себепсіз	Жарамсыз (extreme_no_reason)
Себеп 3 таңбадан қысқа	Жарамсыз (reason_too_short)
Бір таңбаның қайталануы	Жарамсыз (gibberish)
Мағынасыз қысқа сөз	Жарамсыз (low_quality)
Қалыпты пікір	Жарамды

2.10 Тестілеу

Әзірленген жүйенің сапасын қамтамасыз ету үшін тестілеудің бірнеше түрі қолданылды. Тестілеудің мақсаты – функциялардың дұрыс жұмыс істейтінін, қауіпсіздік талаптарының сақталғанын және рөлдік шектеулердің бұзылмайтынын тексеру.

Бірінші түрі – ұштан-ұшқа (end-to-end) автоматты тестілеу. Ол Playwright құралымен орындалды және браузердегі нақты пайдаланушы әрекеттерін имитациялайды: кіру, тәрбие сағатын құру, бекіту, есепті көру. Тестілер үш бағытты қамтыды: сайттың толық жұмысы (full-site), рөлдік қол жеткізу (role-access) және қауіпсіздік (security).

Екінші түрі – қолмен (manual) функционалдық тестілеу. Әрбір негізгі сценарий қолмен тексерілді: дұрыс және қате деректер енгізу, рұқсат етілмеген әрекеттерді орындауға тырысу, шекаралық жағдайлар. Үшінші түрі – қауіпсіздік тексерістері: бөгде мекеменің деректеріне қол жеткізу әрекеттері, токенсіз сұраныстар және рөлдерді айналып өту әрекеттері.

Тестілеудің негізгі сценарийлері мен нәтижелері 2.7-кестеде жинақталған. Барлық негізгі сценарийлер сәтті өтті, ал анықталған кемшіліктер түзетілді.

2.7-кесте – Тестілеу сценарийлері мен нәтижелері

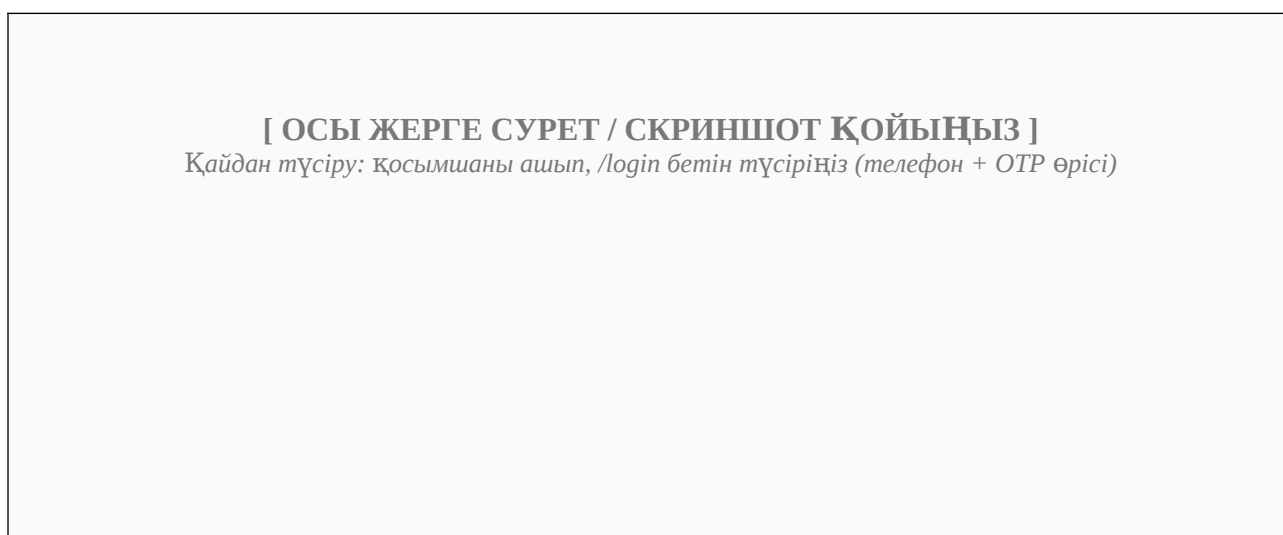
Сценарий	Күтілетін нәтиже	Нәтиже
ОТР арқылы кіру	JWT-токен беріледі	Сәтті
Студент сабақ жасауы	Тыйым салынады (403)	Сәтті

Тәрбие сағатын құру	Сабақ дерекқорға жазылады	Сәтті
Құжатты бекіту	Қолтаңба ендіріледі	Сәтті
Бөгде мекеме деректері	Қол жеткізу жабылады	Сәтті
Айлық есеп	Дұрыс орташа баға	Сәтті

2.11 Жүйенің пайдаланушы интерфейсі (скриншоттар)

Бұл тармақта жүйенің негізгі интерфейс экрандарының скриншоттары келтіріледі. Әрбір сурет жүйенің нақты бетінен түсіріледі. Төмендегі орындарға тиісті скриншоттарды қойыңыз; әр суреттің астында оның қайдан түсірілетіні көрсетілген.

Жүйеге кіру беті пайдаланушыдан телефон нөмірін сұрайды, содан кейін Telegram арқылы келген біржолғы кодты (OTP) растайды (2.5-сурет).



2.5-сурет – Жүйеге кіру (авторизация) беті

Жүйеге кіргеннен кейін пайдаланушыға рөліне сәйкес басты панель (dashboard) ашылады (2.6-сурет).



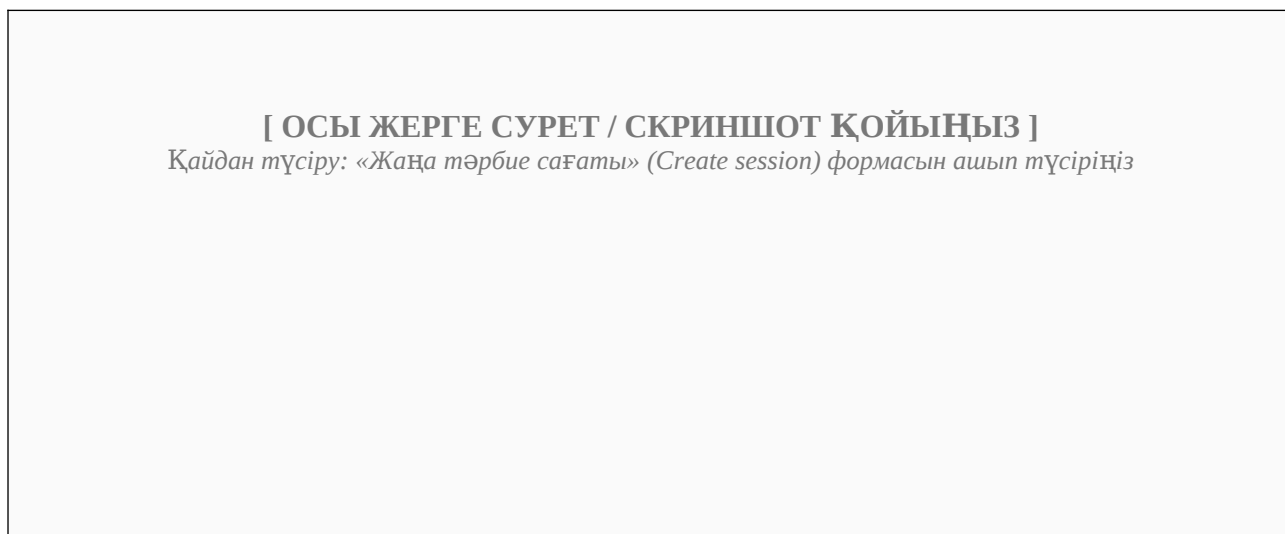
2.6-сурет – Әкімшінің басты беті (dashboard)

Тәрбие сағаттары бөлімінде куратор өзінің жоспарланған және өткізілген сабақтарының тізімін көреді (2.7-сурет).



2.7-сурет – Тәрбие сағаттарының тізімі

Жаңа тәрбие сағатын құру формасында тақырып, күн, сынып, аудитория және уақыт көрсетіледі (2.8-сурет).



2.8-сурет – Тәрбие сағатын құру формасы

Сабақ өткізілгеннен кейін куратор қатысуды белгілейді: Әр студенттің қатысқан-қатыспағанын тіркейді (2.9-сурет).

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]
Қайдан түсіру: сабақты ашып, қатысуды белгілеу (Attendance) терезесін түсіріңіз

2.9-сурет – Қатысуды белгілеу терезесі

Құжатты бекіту бөлімінде әкімші күтіп тұрған құжаттарды қарап, оларды электронды қолтаңбамен бекітеді. Қолтаңба алаңында (SignaturePad) пайдаланушы қолын салады (2.10-сурет).

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]
Қайдан түсіру: «Сабақты бекіту» (Lesson approvals) бетін және қолтаңба терезесін түсіріңіз

2.10-сурет – Құжатты бекіту және электронды қолтаңба алаңы

Студент өткізілген тәрбие сағатына баға беріп, пікір қалдыра алады (2.11-сурет).

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]
Қайдан түсіру: студент ретінде кіріп, рейтинг беру терезесін түсіріңіз

2.11-сурет – Студенттің сабаққа баға беру (рейтинг) терезесі

Telegram бот пайдаланушымен сұхбат арқылы әрекеттеседі: аккаунтты байланыстыру, сабақтарды көру және растау (2.12-сурет).

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]
Қайдан түсіру: Telegram-да ботпен сұхбатты (/start, /my_sessions) түсіріңіз

2.12-сурет – Telegram боттың сұхбат терезесі

Жүйенің сапасын тексеру үшін автоматты тестілер орындалады; олардың нәтижесі тестілеу панелінде көрінеді (2.13-сурет).

[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]
Қайдан түсіру: тестілеу панелін (немесе Playwright есебін) түсіріңіз

2.13-сурет – Автоматты тестілеу нәтижелерінің панелі

2.12 Орнату және пайдаланушы нұсқаулығы

Жүйені орналастыру (deploy) Cloudflare құралдары арқылы жүргізіледі. Алдымен D1 деректер қоры, KV кеңістігі және Queues кезегі құрылады. Содан кейін көші-қон сценарийлері (migrations) іске қосылып, кестелер жасалады. Құпия кілттер (JWT кілті, Telegram бот токени) wrangler құралы арқылы қауіпсіз сақталады. Соңында Workers пен веб-қосымша Cloudflare-ге жарияланады. Бүкіл процесс GitHub Actions арқылы автоматтандырылған: main тармағына өзгеріс енгізілгенде барлық құрауыштар автоматты түрде орналастырылады.

Пайдаланушы нұсқаулығы. Жүйемен жұмыс істеу үшін пайдаланушыға тек заманауи браузер қажет. Кіру беті арқылы пайдаланушы телефон нөмірін енгізеді, Telegram арқылы келген біржолғы кодты растайды және жүйеге кіреді. Кіргеннен кейін рөлге сәйкес интерфейс ашылады.

Куратор «Тәрбие сағаттары» бөлімінде жаңа сабақ құрады: тақырыпты, күнді, сыныпты, аудитория мен уақытты көрсетеді. Сабақ өткізілгеннен кейін оны аяқталды деп белгілейді, қатысуды тіркейді және құжатты бекітуге жібереді.

Әкімшінің нұсқаулығы. Әкімші «Сабақты бекіту» бөлімінде күтіп тұрған құжаттарды қарап, оларды бекітеді немесе қайтарады. Сондай-ақ ол пайдаланушылар мен сыныптарды басқарады, есептерді көреді. Студент өзіне қатысты сабақтарды, бағаларды көреді және өткізілген сабаққа рейтинг бере алады.

2.13 Жүйенің қауіпсіздігі

Қауіпсіздік бірнеше деңгейде қамтамасыз етіледі. Аутентификация JWT-токенмен жүреді, барлық жабық маршруттар токенмен қорғалған. Рөлдік модель пайдаланушылардың тек өз құзыретіндегі әрекеттерді орындауын қамтамасыз етеді. Деректер мекемелер бойынша оқшауланған. Кіріс деректер

Zod схемалары арқылы тексеріледі, сұраныс жиілігі шектеледі (rate limiting), ал сұраныс денесінің көлемі шектеулі. Telegram webhook құпия токенмен расталады.

Осылайша, жүйеде ақпараттық қауіпсіздіктің негізгі қағидаттары – құпиялылық, тұтастық және қолжетімділік – сақталады. Бұл пайдаланушы деректерінің қорғалуын және жүйенің сенімді жұмысын қамтамасыз етеді.

2.14 Жобаны жетілдіру перспективалары

Әзірленген жүйе толыққанды жұмыс істейтін өнім болғанымен, оны әрі қарай дамытудың бірнеше бағыты бар. Архитектураның модульдік болуы жаңа мүмкіндіктерді бар кодты бұзбай қосуға мүмкіндік береді.

- ата-аналарға арналған жеке кабинет пен хабарлама арнасын қосу;
- мобильді қосымша (iOS/Android) әзірлеу;
- тәрбие жұмысы бойынша кеңейтілген аналитика мен бақылау тақтасын (dashboard) қосу;
- тәрбие сағаттарының тақырыптық жоспарын (контент кітапханасын) енгізу;
- есептерді PDF және Excel форматтарына экспорттау мүмкіндігін кеңейту;
- білім беру басқармасының жүйелерімен интеграция орнату.

Бұл бағыттар жүйенің функционалдық мүмкіндіктерін кеңейтіп, оны нақты оқу орындарының тәжірибесіне толық енгізуге жол ашады.

3 ЭКОНОМИКАЛЫҚ БӨЛІМ

Экономикалық бөлімде «Тәрбие Сағаты» ақпараттық жүйесін әзірлеудің еңбек қарқындылығы, еңбекақы шығындары, электр энергиясы мен өзге шығындар есептеліп, жобаның экономикалық тиімділігі бағаланады. Барлық есептеулер мен формулалар төменде толық келтірілген.

3.1 Еңбек қарқындылығын анықтау

Дипломдық жобаның еңбек қарқындылығын анықтау бірқатар шығындардан тұрады:

$$T = t_u + t_a + t_n + t_{отл} + t_d, (1)$$

- t_u - Алгоритмді зерттеуге арналған еңбек шығындары;
- t_a - Алгоритмнің блок-схемасын жасау шығындары;
- t_n - Бағдарламалау шығындары;
- $t_{отл}$ - Компьютерлік бағдарламаны өңдеуге кеткен шығындар;
- t_d - Тапсырма бойынша құжаттаманы дайындау шығындары.

Бағдарламашы сипаттамасы мен біліктілігін ескере отырып, тапсырманың сипаттамасын зерделеу үшін еңбек шығындары адам-сағатпен өлшенеді және формула бойынша анықталады

$$t_u = \frac{Q * B}{(75 \dots 85) * K}$$

, (2)

- Q –Шартты операторлар саны;
- B –Шығынның өсу коэффициенті;
- K –Әзірлеуші біліктілік коэффициенті (бағдарламашының еңбек өтіліне байланысты).

Еңбек шығындарының компоненттерін бағдарламалық өнімдегі операторлардың шартты санын анықтау арқылы анықтауға болады. Оған міндеттерді орындау барысында бағдарламашымен қарастыру қажет, проблеманы белгілеу және алгоритмді жетілдіруді қажет ететін операторлар кіреді. Бағдарламада операторлардың шартты саны (Q) келесі формула бойынша анықталады:

$$Q = q * c * (1 + p), (3)$$

- q – операторлардың болжамды саны;
- c – Бағдарламаның күрделілік коэффициенті (1...2);
- p – дамыту кезінде бағдарламаны түзету коэффициенті(0,5...1).

Білім берудің компьютерлік бағдарламаларын әзірлеу кезінде операторлардың болжамды саны (q) білім беру процесінде бағдарламалық өнімдерді пайдаланатын студенттер, оқытушылар, техниктер, лаборанттардың санын қамтиды.

Әзірлеушілердің еңбек шығындарын есептеу үшін орташа мәндерді алуға болады

- операторлардың болжамды саны $q=(130.....170)$;
- күрделілік коэффициенті $c= 1,5$;
- түзету коэффициенті $p=0,75$.

Осы шарттар арқылы, (3) формуланы пайдаланып, шартты операторлардың санын анықтаймыз **Q**:

$$Q = 130 * 1,5 * (1 + 0,75) = 228 * 1,75 = 341 \text{ адам.}$$

B –Бұл коэффициент 1,2-ден 5-ке дейінгі мәндерді қабылдай алады (орташа мәнді қабылдауға болады). Орташа мәнін алыңыз **B-1,8**

K – әзірлеуші біліктілік коэффициенті осы бағдарламалық жасақтама өнімімен бағдарламашы тәжірибесіне байланысты. Біліктілік коэффициенті қызметтің ұзақтығына байланысты дискретті мәндерді алады:

- а) екі жылға дейін $K=0,8$;
- б) екі жылдан үш жылға дейін $K=1$;
- в) үш жылдан жеті жылға дейін $K=1,3...1,4$;
- г) жеті жылдан жоғары $K=1,5...1,6$.

Біз формуладағы таңдалған мәндерді алмастыратын бағдарламашы сипаттамасын және біліктілігін түсіндіруді ескере отырып, мәселенің сипаттамасын зерделеу үшін еңбек шығындарын есептейміз (2):

адам/сағат

t_a –проблеманы шешу алгоритмін әзірлеуге арналған еңбек шығындары формула бойынша есептеледі:

$$t_a = \frac{Q * B}{(60...75) * K}, (4)$$

Q – шартты операторлар саны;

K – әзірлеуші біліктілік коэффициенті (20);

t_n – Бағдарламалауға қажетті уақыт дербес фотосуретпен анықталады және шамамен 20-30% жалпы еңбек шығынынан.

$$t_a = (341 * 1,8) / (75 * 0,8) = 614 / 60 = 10 \text{ адам/сағат}$$

Дербес компьютердегі бағдарламаны автономды күтіп тұрған бір міндеті бар отладка шығындары формула бойынша анықталады:

$$t_{отл(авт)} = \frac{Q * B}{(40 \dots 50) * K}, (5)$$

Q – шартты операторлар саны;

K – әзірлеуші біліктілік коэффициенті (40);

$$t_{отл(авт)} = (341 * 1,8) / 40 = 614 / 40 = 15 \text{ адам/сағат}$$

Бағдарламаны күрделі жөндеуден кейін түзету шығындары 1,5 есе артады. Осылайша, біз соңғы шығындарды анықтаймыз

$$T_{отл} = 1,5 * t_{отл(авт)}, (6)$$

$$T_{отл} = 1,5 * 15 = 22 \text{ адам/сағат}$$

Тапсырмаға құжаттаманы дайындау бойынша еңбек шығындары қолжазбаға материалдар дайындау үшін еңбек шығындарынан және редакциялау құнын құрайды, құжаттарды басып шығару және орындау келесі формуламен анықталады

$$T_d = t_{дп} + t_{др}, (7)$$

$t_{дп}$ – қолжазбаға материалдар дайындау үшін кеткен еңбек шығындары;

$t_{др}$ - редакциялауға, басып шығаруға және құжаттамаға жұмсалатын шығындар.

Қолжазбаға материалдар дайындау үшін еңбек шығындары формула бойынша анықталады:

$$t_n = \frac{Q * B}{(150 \dots 200) * K}, (8)$$

Q – шартты операторлар саны;

K – әзірлеуші біліктілік коэффициенті (160);

$$td_n = (341 * 1,8) / 160 = 614 / 160 = 4 \text{ адам/сағат}$$

Редакциялауға, басып шығаруға және құжаттамаға жұмсалған шығындар қолжазбаның материалдарын дайындау шығындарымен тікелей пропорционалды.

$$t_p = t_{дп} * 0,75, (9)$$

Жоғарыда келтірілген формулаға қолжазбаға материалдар дайындау шығындарының есептелген құны құжаттарды өңдеу, өңдеу және басып шығару шығындарын анықтайтын болады:

$$t_p = 4 * 0,75 = 3 \text{ адам/сағат}$$

(7) формула бойынша құжаттаманы дайындауға кеткен еңбек шығындарды анықтаймыз:

$$T_d = 4 + 3 = 7 \text{ адам/сағат}$$

Есептелген деректерді қолдану арқылы формуланың дамуының жалпы күрделілігін анықтаймыз(1), бағдарламалауға жұмсалатын шығындар сағатынаадамды құрайды:

$$T = 10+10+4+22+7=53 \text{ адам/сағат}$$

3.2 Еңбек шығындары

Жалақыны есептеу формуласы (10,11)

№

Шегерімдер

Формула

Оклад =ЕА=

1

ЗТ (Міндетті зейнетақы төлемдері)

$$ЗТ=ЕА *10\%$$

15 000 тг

2

ЖТС (Жеке табыс салығы)

$$ЖТС=(ЗПЛ-ОПВ-Шегеру(1 \text{ айлық көрсеткіш}))*10\%$$

13 107 тг

3

ӨА (Өлеуметтік аударымдар)

$$ӨА=(ЕА-ЗТ)*5\%$$

6 750 тг

4

ӨС (Өлеуметтік салық)

$$ӨС=((ЕА-ЗТ)*11\%)-ӨА$$

8 100 тг

5

Қолға алатын еңбек ақы

$$ЕА-ЖТС-ЗТ$$

121 893 тг

6

Жұмыс берушілердің шығындары

$$ЕА+ӨС+ӨЕ$$

164 850 тг

Осылайша, жалақы қоры 150 000 теңге құрайды

3.3 Бағдарламаны жасау және түзету кезінде кеткен уақыт шығындарын есептеу

Бағдарламаны жасау және түзету кезінде компьютер уақытын төлеу шығындары нақты уақытты көбейту және бағдарламаны компьютерлік жабдықтың машина сағатының бағасы бойынша жөндеу арқылы анықталады:

$$C_M = C * (t_n + t_{отл}), \quad (12)$$

t_n – бағдарламалау шығындары;

$t_{отл}$ – бағдарламаның кешіктірілуіне байланысты шығыстар

C – машина сағат бағасы келесі формула бойынша есептеледі:

$$C = \frac{З_{аш} + З_{вм} + З_{тр} + З_{пр}}{T_{пк}}$$

$T_{пк}$

$З_{аш}$ – жылдық амортизациялық шығындары, теңге/жыл;

$З_{вм}$ – қосымша материалдардың жыл сайынғы шығыны, теңге/жыл;

$З_{тр}$ – компьютерге қызмет көрсету шығыны, теңге/жыл;

$З_{пр}$ – басқа да және үстеме шығындарға арналған жылдық шығындар, теңге/жыл;

$T_{пк}$ – нақты жылдық уақыт қоры сағат/жыл;

Амортизацияның жылдық шығындары мынадай формула бойынша анықталады:

$$(14)$$

$C_{бал}$ – компьютердің баланстық құны, теңге / шт.

H_a – амортизация нормасы, %

Компьютердің баланстық құны компьютердің өзіндік құнын және оны жеткізу және орнату шығындарының сомасы ретінде анықталады:

$$C_{бал} = C_{нар} + З_{уст}, \quad (15)$$

$C_{рын}$ – компьютердің нарықтық құны, теңге /шт.;

$З_{уст}$ – тасымалдау және орнату шығындары, теңге /шт.;

Дербес компьютерді жеткізу және орнату құны компьютердің 0,02 пайызын құрайды.

Компьютердің нарықтық құны 495000 теңге. Енді компьютердің нарықтық құнын белгілей отырып, жеткізу және орнату шығындарын есептейміз:

$$З_{уст} = 495000 * 0,02 = 9900 \text{ теңге.}$$

Компьютердің баланстық құнын формула арқылы анықтаңыз(15):

$$C_{бал} = 495000 + 9900 = 504900 \text{ теңге.}$$

Амортизацияның жыл сайынғы құны, баланстық құнды ескере отырып және амортизация нормасын ескере отырып, формула бойынша анықталады 40%:

$$Z_{ам} = 495000 * 40\% = 198000 \text{ теңге.}$$

Қызметкерлердің жалақыларына жұмсалатын жыл сайынғы шығындар қарастырылмайды, өйткені біз бір компьютердің қатысуын жалғастырамыз және бір дербес компьютер үшін қызметкерлерді ұстап тұру экономикалық жағынан тиімді емес.

Қосалқы материалдардың жыл сайынғы құны компьютердің баланстық құнының 1 пайызын құрайды.

$$Z_{см} = C_{бал} * 0,01 = 495000 * 0,01 = 4950 \text{ теңге.}$$

Компьютердің есептік жазбасының ағымдағы құнының 5 пайызына ағымдағы жөндеу және жаңарту (Жаңарту) бойынша шығындар (кейбір жағдайларда ол әлдеқайда жоғары болуы мүмкін).

$$Z_{тр} = C_{бал} * 0,05 = 495000 * 0,05 = 24750 \text{ теңге.}$$

Басқа және үстеме шығыстарға жұмсалатын шығындар коммуналдық қызметтерге, коммуникацияларға, жалдау ақысына және т.б. төлеуге арналған шығыстарды қамтиды және компьютердің баланстық құнының 5% -ын құрайды

$$Z_{пр} = C_{бал} * 0,05 = 495000 * 0,05 = 24750 \text{ теңге.}$$

Дербес компьютердің жылдық нақты уақыттық қоры жылына сағат бойынша өлшенеді және формула бойынша анықталады:

$$T_{пк} = N_m * N_d * N_q, \text{ (16)}$$

где N_m – жылдағы айлар саны, (12 ай./жыл);

N_d – жұмыс күндерінің орташа саны, (22 күн./ай.);

N_q – орташа жұмыс күні (8 сағат/күн).

Осылайша, дербес компьютердің нақты жылдық қоры:

$$T_{пк} = 12 * 22 * 8 = 2112 \text{ час/год}$$

Формуланы пайдаланып, бағдарламаны жазу және түзету кезінде компьютер уақытын төлеудің шығындарын есептеңіз (13).

Енді формула бойынша машина сағатының бағасын есептеуге арналған барлық деректер бар (13):

$$C = \underline{3аш + 3вм + 3гр + 3пр}$$

Тпк

теңге /сағ

$$C = \frac{198000 + 4950 + 24750 + 24750}{2112} = 120$$

Формуланы пайдаланып, бағдарламаны жасау және түзету кезінде уақыт шығындарын есептейміз (12):

$$C_m = 120 * (4 + 22) = 3120 \text{ теңге}$$

$$C_m = 120 * (4 + 22) = 3120$$

3.4 Электр энергия шығындарын есептеу

Энергия шығыны ($C_э$) бағдарламаны жөндеу кезінде электр энергиясының құны анықталады (C_c), тұтынылатын қуат (P) және жалпы бағдарламалау уақыты (t_n), жөндеу ($t_{отл}$) және тапсырмаға құжаттаманы дайындау (t_d). Электр энергиясын төлеу жөніндегі шығындар формула ретінде жазылуы мүмкін:

$$C_э = C_c * P * (t_n + t_{отл} + t_d), \quad (17)$$

C_c – электр энергиясының құны;

P – бір дербес компьютермен тұтынылатын қуат;

t_n – бағдарламалау уақыты;

$t_{отл}$ – жөндеу уақыты;

t_d – тапсырманы дайындау уақыты.

Дербес компьютер тұтынатын қуат монитордың тұтынатын қуатынан және жүйелік блокпен тұтынылатын қуаттан тұрады. Осылайша, компьютердің қуаты **1,2 кВт**.

(17) Формуланы пайдалану арқылы электр энергиясының құнын анықтаймыз **30,17** теңге /кВт:

$$C_э = 1,2 * 30,17 * (4 + 22 + 7) = 1195 \text{ теңге.}$$

3.5 Басқа да шығындар есебі

Басқа да шығындар бағдарламаны электронды немесе қағаз тасымалдағыштағы репликациялаумен байланысты және жалпы шығындардың 5-20% -ын құрайды (пайдаланушылардың санына байланысты)

$$C_{пр} = (T + EA + \Theta C + CM) * 0,05 =$$

$$C_{пр} = (53 + 150000 + 8100 + 3120) * 0,05 = 8064 \text{ теңге.}$$

$$Зобщ = T + EA + \Theta C + CM + C_{пр} =$$

$$Жшығындар = 53 + 150000 + 8100 + 3120 + 1195 + 8064 = 170532 \text{ теңге.}$$

Бағдарламалық жасақтаманы әзірлеуге жұмсалатын шығындар

170 532 тг

Шығындар индексі

Сумма, теңге.

1 Бағдарламашының еңбек ақысы

ЕА

150 000

2 Бірыңғай Әлеуметтік салық

ӘС

8 100

3 Бағдарламаны жөндеуге кететін уақытты төлеу шығындары

С_м

3120

4 Бағдарламаны жөндеу кезінде электр энергиясына ақы төлеу

С_э

1195

5 Басқа шығындар

С_{пр}

8 064

Барлығы:

Бшығындар

170 479

Бағдарламалық өнімге жұмсалатын шығындар 170 479

3.6 Пайдаланудың экономикалық тиімділігінің көрсеткіштерін есептеу

Экономикалық тиімділік – кәсіпорынның қаржылық қызметінің жалпылама көрсеткіші. Шаруашылықты жүргізудің түпкілікті нәтижелерін пайдадан гөрі неғұрлым толық сипаттайды, өйткені оның шамасы нәтижеліліктің қолда бар және пайдаланылған ресурстармен арақатынасын көрсетеді. Тиімділік көрсеткіштері кәсіпорынның қаржылық қызметін бағалау үшін және инвестициялық саясат пен баға белгілеу үшін қолданылады. X

Ақпаратты өңдеу жүйелерін автоматтандыру саласындағы экономикалық тиімділік нәтижеге қол жеткізу, нәтижелерді құжаттау, осы

жұмыстарды жүргізуді жеделдету, құжаттағы қателер мен кемшіліктерді азайту, алынған шешімдерді оңтайландыру үшін қажетті еңбек шығындарын төмендетумен байланысты.

Келесі формула бойынша жылдық экономикалық тиімділікті анықтауға болады

Э т жылдық= Жшығындар - Бшығындар , (18)

Эт = 170 479-170532=53

Осылайша, бұл жобаны іске асырудың экономикалық нәтижесі шығынды үнемдеу болады, яғни бағдарламалық жасақтама тапсырыс беруші үшін экономикалық тиімді деп танылуы мүмкін.

Бағдарламалық өнімдердің тар және жергілікті мақсаттарға арналған экономикалық тиімділігін бағалау кезінде, мысалы, оқу үдерісінде, жоғарыда келтірілген әдіспен есептелген экономикалық әсер теріс мәнге ие болуы мүмкін

4 ЕҢБЕКТИ ҚОРҒАУ ЖӘНЕ ҚАУІПСІЗДІК

Бағдарламалық қамтамасыз етуді әзірлеу негізінен компьютер алдында отырып орындалатын ой еңбегі болып табылады. Мұндай жұмыс физикалық тұрғыдан ауыр болмаса да, адамның көру мүшелеріне, тірек-қимыл аппаратына және жүйке жүйесіне айтарлықтай жүктеме түсіреді. Сондықтан еңбек қауіпсіздігі мен өндірістік санитария талаптарын сақтау жобаны сапалы әрі қауіпсіз орындаудың маңызды шарты болып табылады.

«Тәрбие Сағаты» жобасын әзірлеу кезінде негізгі қауіптер экран алдындағы еңбекпен, жарық режимімен, электр жабдығымен және ақпараттық қауіпсіздікпен байланысты болды. Ұйымдастырушылық және техникалық талаптарды сақтаған жағдайда бұл қауіптер басқарылатын деңгейде болады.

4.1 Жұмыс орнын ұйымдастыру және эргономика

Бағдарламашының жұмыс орны эргономикалық талаптарға сай болуы тиіс. Эргономикалық орындық омыртқаға түсетін жүктемені азайтады, ал биіктігі дұрыс реттелген үстел мен монитор дене қалпын сақтауға көмектеседі. Монитор көзден шамамен 50–70 см қашықтықта, жоғарғы шеті көз деңгейінде немесе сәл төмен орналасуы керек.

Ұзақ уақыт бір қалыпта отыру тірек-қимыл аппаратына қосымша жүктеме түсіреді. Осыған байланысты әрбір 45–60 минут сайын қысқа үзіліс жасап, жеңіл дене жаттығуларын орындау ұсынылады. Бұл қан айналымын жақсартып, шаршауды азайтады.

- эргономикалық орындық пен реттелетін үстел қолдану;
- монитордың дұрыс орналасуы мен қашықтығын сақтау;
- әрбір 45–60 минут сайын үзіліс жасау;
- жұмыс барысында дене қалпын өзгертіп отыру.

4.2 Жарықтандыру және көру қабілетіне түсетін жүктеме

Бағдарламашының негізгі жұмыс құралы – монитор. Ұзақ уақыт экранға қарау көздің құрғауына, көру қабілетінің төмендеуіне және бас ауруына әкелуі мүмкін. Жұмыс орны табиғи және жасанды жарықпен жеткілікті түрде жарықтандырылуы тиіс. Жасанды жарықтандырудың ұсынылатын деңгейі – 300–500 люкс. Монитор бетіне тікелей түсетін жарық пен шағылысуды болдырмау қажет.

Көзге түсетін жүктемені азайту үшін көк жарықты (blue light) сүзетін көзілдіріктерді немесе монитордың тиісті режимдерін қолдануға болады. Сондай-ақ келесі ережелерді сақтаған жөн:

- 20–20–20 ережесін сақтау: әр 20 минут сайын 20 секунд 6 метр қашықтыққа қарау;
- монитордың жарықтығы мен контрастын дұрыс реттеу;
- қаріп өлшемін ыңғайлы деңгейде ұстау;

– жұмыс орнын жеткілікті жарықтандыру.

4.3 Электр қауіпсіздігі

Компьютерлік техника электр желісінен қоректенетіндіктен, электр қауіпсіздігі талаптарын сақтау міндетті. Барлық құрылғылар жерге тұйықталған розеткаларға қосылуы, ал сымдардың оқшаулауы бүтін болуы тиіс. Бір розеткаға бірнеше қуатты құрылғыны қосуға болмайды, себебі бұл шамадан тыс жүктемеге және өрт қаупіне әкеледі.

Желілік кернеудің секірулерінен қорғану үшін желілік сүзгілер (сетевой фильтр) немесе үздіксіз қоректендіру көздерін (UPS) қолданған жөн. Электр құрылғыларын ұзақ уақыт қараусыз қалдыруға болмайды, ал ақаулық байқалған жағдайда құрылғыны дереу токтан ажырату қажет.

4.4 Жұмыс орнындағы микроклимат

Жұмыс орнындағы ауа температурасы мен ылғалдылық деңгейі қызметкердің еңбек өнімділігіне тікелей әсер етеді. Бөлмені үнемі желдетіп отыру қажет, себебі ауадағы шаң мен оттегінің жетіспеуі шаршауды арттырады. Ұсынылатын микроклимат көрсеткіштері 4.1-кестеде келтірілген.

4.1-кесте – Жұмыс орнындағы микроклимат көрсеткіштері

Көрсеткіш	Ұсынылатын мән
Ауа температурасы	+20 ... +24 °C
Салыстырмалы ылғалдылық	40 ... 60 %
Ауа қозғалысының жылдамдығы	0,1 ... 0,2 м/с
Желдету	Жүйелі түрде

4.5 Ақпараттық қауіпсіздік

Бағдарламалық жобаны әзірлеу барысында ақпараттық қауіпсіздік ерекше маңызға ие. Деректердің жоғалуы немесе рұқсатсыз қол жеткізу жобаға елеулі зиян келтіруі мүмкін. «Тәрбие Сағаты» жүйесі пайдаланушылардың жеке деректерін (аты-жөні, телефоны) өңдейтіндіктен, олардың қорғалуы аса маңызды.

Ақпараттық қауіпсіздікті қамтамасыз ету үшін келесі шаралар қолданылды: құпиясөздер мен токендерді қауіпсіз сақтау, деректердің резервтік көшірмесін жасау, антивирустық қорғаныс және жүйелік бағдарламаларды уақытылы жаңарту. Жобаның бастапқы коды нұсқаларды басқару жүйесінде (Git) сақталады, бұл өзгерістерді бақылауға және қажет болса қалпына келтіруге мүмкіндік береді.

- күрделі құпиясөздер мен қауіпсіз токендер қолдану;
- деректердің резервтік көшірмесін жүйелі жасау;
- антивирустық қорғанысты пайдалану;
- бағдарламалық жасақтаманы уақытылы жаңарту.

4.6 Психологиялық аспектілер мен Өрт қауіпсіздігі

Бағдарламашының жұмысы жоғары зейінділікті, логикалық ойлауды және ұзақ уақыт шоғырлануды талап етеді. Уақыт тапшылығы мен күрделі есептер күйзеліс деңгейін арттыруы мүмкін. Психологиялық қауіптердің алдын алу үшін жұмыс пен демалыс режимін сақтау, қысқа үзілістер жасау және физикалық белсенділікті арттыру ұсынылады.

Өрт қауіпсіздігін қамтамасыз ету мақсатында жұмыс орнында өрт сөндіргіш болуы, авариялық шығу жолдарының бос болуы және электр құрылғыларын дұрыс пайдалану қажет. Төтенше жағдай (Өрт немесе электр ақауы) туындаған кезде жабдықты токтан ажырату, қауіпсіз жерге шығу және тиісті қызметтерге (101 немесе 112) хабарлау керек.

4.7 Қорытынды

«Тәрбие Сағаты» жүйесін әзірлеу барысында еңбек қауіпсіздігі, өндірістік санитария және ақпараттық қауіпсіздік талаптары толық сақталды. Жұмыс негізінен компьютер алдында орындалғандықтан, басты назар экран алдындағы еңбекке, жарықтандыруға, электр қауіпсіздігіне және психологиялық жүктемеге аударылды. Қауіпсіздік талаптарын сақтау қызметкердің денсаулығын қорғауға, еңбек өнімділігін арттыруға және бағдарламалық өнімді сапалы әзірлеуге жол ашады.

ҚОРЫТЫНДЫ

Осы дипломдық жобада оқу орнындағы тәрбие сағаттарын электронды түрде бекіту және басқаруға арналған «Тәрбие Сағаты» атты веб-негіздегі ақпараттық жүйе әзірленді. Жұмыс барысында пәндік аймақ зерттелді, қолданыстағы шешімдерге шолу жасалды, функционалдық және функционалдық емес талаптар анықталды, клиент–серверлік архитектура жобаланды, деректер қоры құрылды, серверлік және клиенттік бөліктер іске асырылды, тестілеу жүргізілді, экономикалық және еңбек қауіпсіздігі бөлімдері дайындалды.

Жұмыстың негізгі нәтижесі – толыққанды жұмыс істейтін ақпараттық жүйе. Жүйе тәрбие сағатын жоспарлауды, бекітуді, қатысуды есепке алуды, Word-құжатты электронды қолтаңбамен бекітуді, Telegram арқылы хабарламаларды, AI-көмекшіні және ай сайынғы есептерді бір ортаға біріктіреді. Жүйе үш рөлге – әкімшіге, кураторға және студентке – арналған рөлдік интерфейстерді қамтиды.

Жобаны әзірлеу барысында ақпараттық жүйелерді жобалау, деректер қорын құру, REST API ұйымдастыру, заманауи веб-интерфейс жасау және бұлтты технологияларды (Cloudflare Workers, D1, KV, Queues) қолдану бойынша теориялық білім мен практикалық дағдылар ұштастырылды. JWT негізіндегі аутентификация, рөлдік қол жеткізу және деректерді оқшаулау арқылы жүйенің қауіпсіздігі қамтамасыз етілді.

Жобаның күшті жақтары: жеңіл әрі заманауи технологиялық стек, серверсіз (serverless) архитектура, көп мекемелік модель, электронды қолтаңбамен құжат бекіту тетігі және екі тілді қолдау. Болашақта жүйеге ата-аналар кабинетін, мобильді қосымшаны, кеңейтілген аналитиканы және басқа жүйелермен интеграцияны қосу арқылы оны әрі қарай дамытуға болады.

Қорытындылай келе, әзірленген «Тәрбие Сағаты» жүйесі дипломдық жоба талаптарына сай келетін, функционалды, логикалық түрде аяқталған және нақты оқу орындарында қолдануға жарамды өнім ретінде бағаланады. Жобаның алдына қойылған барлық міндеттер толық орындалды.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР ТІЗІМІ

- 1 Қазақстан Республикасының «Білім туралы» Заңы. – Астана, 2007 (өзгертулерімен).
- 2 Қазақстан Республикасының «Дербес деректер және оларды қорғау туралы» Заңы. – Астана, 2013.
- 3 TypeScript Documentation. – [Электрондық ресурс]. URL: <https://www.typescriptlang.org/docs/> (қаралған күні: 2026).
- 4 Cloudflare Workers Documentation. – [Электрондық ресурс]. URL: <https://developers.cloudflare.com/workers/> (қаралған күні: 2026).
- 5 Cloudflare D1 Documentation. – [Электрондық ресурс]. URL: <https://developers.cloudflare.com/d1/> (қаралған күні: 2026).
- 6 Hono Web Framework. – [Электрондық ресурс]. URL: <https://hono.dev/> (қаралған күні: 2026).
- 7 React Documentation. – [Электрондық ресурс]. URL: <https://react.dev/> (қаралған күні: 2026).
- 8 Vite. Next Generation Frontend Tooling. – [Электрондық ресурс]. URL: <https://vitejs.dev/> (қаралған күні: 2026).
- 9 Tailwind CSS Documentation. – [Электрондық ресурс]. URL: <https://tailwindcss.com/docs> (қаралған күні: 2026).
- 10 Zod. TypeScript-first schema validation. – [Электрондық ресурс]. URL: <https://zod.dev/> (қаралған күні: 2026).
- 11 SQLite Documentation. – [Электрондық ресурс]. URL: <https://www.sqlite.org/docs.html> (қаралған күні: 2026).
- 12 JSON Web Tokens (RFC 7519). – [Электрондық ресурс]. URL: <https://datatracker.ietf.org/doc/html/rfc7519> (қаралған күні: 2026).
- 13 Telegram Bot API. – [Электрондық ресурс]. URL: <https://core.telegram.org/bots/api> (қаралған күні: 2026).
- 14 Google Gemini API Documentation. – [Электрондық ресурс]. URL: <https://ai.google.dev/> (қаралған күні: 2026).
- 15 Қазақстан Республикасының Еңбек кодексі. – Астана, 2015 (өзгертулерімен).
- 16 ГОСТ 12.0.003-2015. Опасные и вредные производственные факторы. Классификация. – М.: Стандартиформ, 2015.

Деректер қорының схемасы (D1, SQLite)

Төменде жүйенің деректер қорын құратын негізгі көші-қон сценарийі (migration) келтірілген. Ол кестелерді, шетелдік кілттерді, ЧЕКС-шектеулерін және индекстерді жасайды.

```
-- Тәрбие Сағаты Manager – D1 Migration  
-- Complete schema with indexes and foreign keys
```

```
CREATE TABLE IF NOT EXISTS schools (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  bin TEXT NOT NULL UNIQUE,  
  city TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

```
CREATE INDEX idx_schools_city ON schools(city);  
CREATE UNIQUE INDEX idx_schools_bin ON schools(bin);
```

```
CREATE TABLE IF NOT EXISTS users (  
  id TEXT PRIMARY KEY,  
  school_id TEXT NOT NULL,  
  full_name TEXT NOT NULL,  
  role TEXT NOT NULL CHECK(role IN ('admin', 'teacher', 'student', 'parent')),  
  phone TEXT NOT NULL,  
  telegram_chat_id TEXT,  
  lang TEXT NOT NULL DEFAULT 'ru' CHECK(lang IN ('kz', 'ru')),  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  FOREIGN KEY (school_id) REFERENCES schools(id) ON DELETE CASCADE  
);
```

```
CREATE INDEX idx_users_school_id ON users(school_id);  
CREATE INDEX idx_users_role ON users(role);  
CREATE INDEX idx_users_phone ON users(phone);  
CREATE INDEX idx_users_telegram_chat_id ON users(telegram_chat_id);
```

```
CREATE TABLE IF NOT EXISTS classes (  
  id TEXT PRIMARY KEY,  
  school_id TEXT NOT NULL,  
  name TEXT NOT NULL,  
  teacher_id TEXT NOT NULL,  
  academic_year TEXT NOT NULL,  
  FOREIGN KEY (school_id) REFERENCES schools(id) ON DELETE CASCADE,  
  FOREIGN KEY (teacher_id) REFERENCES users(id) ON DELETE RESTRICT  
);
```

```
CREATE INDEX idx_classes_school_id ON classes(school_id);  
CREATE INDEX idx_classes_teacher_id ON classes(teacher_id);  
CREATE INDEX idx_classes_academic_year ON classes(academic_year);
```

```
CREATE TABLE IF NOT EXISTS class_students (  
  id TEXT PRIMARY KEY,  
  class_id TEXT NOT NULL,  
  student_id TEXT NOT NULL,  
  FOREIGN KEY (class_id) REFERENCES classes(id) ON DELETE CASCADE,  
  FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,  
  UNIQUE(class_id, student_id)  
);
```

```
CREATE INDEX idx_class_students_class_id ON class_students(class_id);  
CREATE INDEX idx_class_students_student_id ON class_students(student_id);
```

```
CREATE TABLE IF NOT EXISTS tarbie_sessions (  

```

```

    id TEXT PRIMARY KEY,
    class_id TEXT NOT NULL,
    teacher_id TEXT NOT NULL,
    topic TEXT NOT NULL,
    planned_date TEXT NOT NULL,
    actual_date TEXT,
    status TEXT NOT NULL DEFAULT 'planned' CHECK(status IN
('planned', 'completed', 'cancelled', 'rescheduled')),
    duration_minutes INTEGER NOT NULL DEFAULT 45,
    notes TEXT,
    attachment_url TEXT,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (class_id) REFERENCES classes(id) ON DELETE CASCADE,
    FOREIGN KEY (teacher_id) REFERENCES users(id) ON DELETE RESTRICT
);

CREATE INDEX idx_tarbie_sessions_class_id ON tarbie_sessions(class_id);
CREATE INDEX idx_tarbie_sessions_teacher_id ON tarbie_sessions(teacher_id);
CREATE INDEX idx_tarbie_sessions_planned_date ON tarbie_sessions(planned_date);
CREATE INDEX idx_tarbie_sessions_status ON tarbie_sessions(status);
CREATE INDEX idx_tarbie_sessions_class_date ON tarbie_sessions(class_id, planned_date);

CREATE TABLE IF NOT EXISTS session_attendance (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL,
    student_id TEXT NOT NULL,
    status TEXT NOT NULL CHECK(status IN ('present', 'absent', 'late', 'excused')),
    marked_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (session_id) REFERENCES tarbie_sessions(id) ON DELETE CASCADE,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    UNIQUE(session_id, student_id)
);

CREATE INDEX idx_session_attendance_session_id ON session_attendance(session_id);
CREATE INDEX idx_session_attendance_student_id ON session_attendance(student_id);

CREATE TABLE IF NOT EXISTS notifications_log (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    session_id TEXT,
    message_text TEXT NOT NULL,
    sent_at TEXT NOT NULL DEFAULT (datetime('now')),
    status TEXT NOT NULL DEFAULT 'pending' CHECK(status IN
('sent', 'failed', 'pending', 'dead_letter')),
    error_msg TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (session_id) REFERENCES tarbie_sessions(id) ON DELETE SET NULL
);

CREATE INDEX idx_notifications_log_user_id ON notifications_log(user_id);
CREATE INDEX idx_notifications_log_session_id ON notifications_log(session_id);
CREATE INDEX idx_notifications_log_status ON notifications_log(status);
CREATE INDEX idx_notifications_log_sent_at ON notifications_log(sent_at);

CREATE TABLE IF NOT EXISTS notification_templates (
    id TEXT PRIMARY KEY,
    school_id TEXT NOT NULL,
    event_type TEXT NOT NULL CHECK(event_type IN (
    'SESSION_PLANNED', 'SESSION_REMINDER', 'SESSION_COMPLETED',
    'SESSION_RESCHEDULED', 'ABSENCE_ALERT'
)),
    lang TEXT NOT NULL CHECK(lang IN ('kz', 'ru')),
    template_text TEXT NOT NULL,
    FOREIGN KEY (school_id) REFERENCES schools(id) ON DELETE CASCADE,
    UNIQUE(school_id, event_type, lang)
);

```

```

CREATE INDEX idx_notification_templates_school_event ON notification_templates(school_id,
event_type);

-- Seed default school for templates
INSERT INTO schools (id, name, bin, city) VALUES
  ('__default__', 'Default School', '000000000000', 'Default');

-- Seed default notification templates for a placeholder school
-- These will be copied when a new school is created

INSERT INTO notification_templates (id, school_id, event_type, lang, template_text)
VALUES
  ('tpl-default-planned-kz', '__default__', 'SESSION_PLANNED', 'kz',
   'Жаңа тәрбие сағаты жоспарланды!\n\nТақырып: {{topic}}\nКүні: {{date}}\nСынып: {{class_name}}\nМұғалім: {{teacher_name}}'),
  ('tpl-default-planned-ru', '__default__', 'SESSION_PLANNED', 'ru',
   'Запланирован новый классный час!\n\nТема: {{topic}}\nДата: {{date}}\nКласс: {{class_name}}\nУчитель: {{teacher_name}}'),
  ('tpl-default-reminder-kz', '__default__', 'SESSION_REMINDER', 'kz',
   'Ескерту: ертең тәрбие сағаты!\n\nТақырып: {{topic}}\nКүні: {{date}}\nСынып: {{class_name}}'),
  ('tpl-default-reminder-ru', '__default__', 'SESSION_REMINDER', 'ru',
   'Напоминание: завтра классный час!\n\nТема: {{topic}}\nДата: {{date}}\nКласс: {{class_name}}'),
  ('tpl-default-completed-kz', '__default__', 'SESSION_COMPLETED', 'kz',
   'Тәрбие сағаты аяқталды!\n\nТақырып: {{topic}}\nСынып: {{class_name}}\nКелгендер: {{attendance_count}}/{{total_students}}'),
  ('tpl-default-completed-ru', '__default__', 'SESSION_COMPLETED', 'ru',
   'Классный час завершён!\n\nТема: {{topic}}\nКласс: {{class_name}}\nПрисутствовали: {{attendance_count}}/{{total_students}}'),
  ('tpl-default-rescheduled-kz', '__default__', 'SESSION_RESCHEDULED', 'kz',
   'Тәрбие сағаты ауыстырылды!\n\nТақырып: {{topic}}\nЕскі күн: {{old_date}}\nЖаңа күн: {{new_date}}\nСынып: {{class_name}}'),
  ('tpl-default-rescheduled-ru', '__default__', 'SESSION_RESCHEDULED', 'ru',
   'Классный час перенесён!\n\nТема: {{topic}}\nСтарая дата: {{old_date}}\nНовая дата: {{new_date}}\nКласс: {{class_name}}'),
  ('tpl-default-absence-kz', '__default__', 'ABSENCE_ALERT', 'kz',
   'Қатысу ескертуі!\n\n{{student_name}} соңғы 3 тәрбие сағатына қатыспады.\nСынып: {{class_name}}\nМұғалімге хабарласыңыз.'),
  ('tpl-default-absence-ru', '__default__', 'ABSENCE_ALERT', 'ru',
   'Уведомление о посещаемости!\n\n{{student_name}} пропустил(а) последние 3 классных часа.\nКласс: {{class_name}}\nСвяжитесь с классным руководителем.');
```

Аутентификация модулі (JWT)

JWT-токенге қол қою және оны тексеру Workers-тің кірістірілген Web Crypto API-і арқылы, сыртқы кітапханаларсыз орындалады.

lib/jwt.ts

```
import type { JwtPayload } from '@tarbie/shared';

const encoder = new TextEncoder();

async function importKey(secret: string): Promise<CryptoKey> {
  return crypto.subtle.importKey(
    'raw',
    encoder.encode(secret),
    { name: 'HMAC', hash: 'SHA-256' },
    false,
    ['sign', 'verify']
  );
}

function base64UrlEncode(data: Uint8Array): string {
  let binary = '';
  for (let i = 0; i < data.byteLength; i++) {
    binary += String.fromCharCode(data[i]!);
  }
  return btoa(binary).replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/, '');
}

function base64UrlDecode(str: string): Uint8Array {
  const padded = str.replace(/-/g, '+').replace(/_/g, '/');
  const binary = atob(padded);
  const bytes = new Uint8Array(binary.length);
  for (let i = 0; i < binary.length; i++) {
    bytes[i] = binary.charCodeAt(i);
  }
  return bytes;
}

function encodeJson(obj: unknown): string {
  return base64UrlEncode(encoder.encode(JSON.stringify(obj)));
}

export async function signJwt(payload: Omit<JwtPayload, 'iat' | 'exp'>, secret: string,
  ttlSeconds = 86400): Promise<string> {
  const header = { alg: 'HS256', typ: 'JWT' };
  const now = Math.floor(Date.now() / 1000);
  const fullPayload: JwtPayload = {
    ...payload,
    iat: now,
    exp: now + ttlSeconds,
  };

  const headerB64 = encodeJson(header);
  const payloadB64 = encodeJson(fullPayload);
  const signingInput = `${headerB64}.${payloadB64}`;

  const key = await importKey(secret);
  const signature = new Uint8Array(
    await crypto.subtle.sign('HMAC', key, encoder.encode(signingInput))
  );

  return `${signingInput}.${base64UrlEncode(signature)}`;
}
```

```

export async function verifyJwt(token: string, secret: string): Promise<JwtPayload |
null> {
  const parts = token.split('.');
  if (parts.length !== 3) return null;

  const [headerB64, payloadB64, signatureB64] = parts as [string, string, string];
  const signingInput = `${headerB64}.${payloadB64}`;

  const key = await importKey(secret);
  const signature = base64UrlDecode(signatureB64);

  const valid = await crypto.subtle.verify(
    'HMAC',
    key,
    signature,
    encoder.encode(signingInput)
  );

  if (!valid) return null;

  const payload = JSON.parse(
    new TextDecoder().decode(base64UrlDecode(payloadB64))
  ) as JwtPayload;

  if (payload.exp < Math.floor(Date.now() / 1000)) return null;

  return payload;
}

```

middleware/auth.ts

```

import { createMiddleware } from 'hono/factory';
import type { HonoEnv } from '../env.js';
import { verifyJwt } from '../lib/jwt.js';
import type { Role } from '@tarbie/shared';
import { ERROR_CODES } from '@tarbie/shared';

export const authMiddleware = createMiddleware<HonoEnv>(async (c, next) => {
  const authHeader = c.req.header('Authorization');
  if (!authHeader?.startsWith('Bearer ')) {
    return c.json({ success: false, code: ERROR_CODES.UNAUTHORIZED, message: 'Missing or
invalid Authorization header' }, 401);
  }

  const token = authHeader.slice(7);
  const payload = await verifyJwt(token, c.env.JWT_SECRET);
  if (!payload) {
    return c.json({ success: false, code: ERROR_CODES.UNAUTHORIZED, message: 'Invalid or
expired token' }, 401);
  }

  c.set('user', {
    id: payload.sub,
    role: payload.role,
    school_id: payload.school_id,
  });

  await next();
});

export function requireRole(...roles: Role[]) {
  return createMiddleware<HonoEnv>(async (c, next) => {

```

```
    const user = c.get('user');
    if (!roles.includes(user.role)) {
      return c.json({ success: false, code: ERROR_CODES.FORBIDDEN, message: 'Insufficient
permissions' }, 403);
    }
    await next();
  });
}
```

Сервердің кіру нүктесі (Hono)

worker.ts файлы барлық API маршруттарын тіркейді, аралық бағдарламаларды (middleware) қосады және cron-тапсырмаларды өңдейді.

worker.ts

```
import { Hono } from 'hono';
import type { HonoEnv, Env } from './env.js';
import { corsMiddleware } from './middleware/cors.js';
import { bodyLimit } from './middleware/body-limit.js';
import authRoutes from './routes/auth.js';
import sessionRoutes from './routes/sessions.js';
import attendanceRoutes from './routes/attendance.js';
import notificationRoutes from './routes/notifications.js';
import adminRoutes from './routes/admin.js';
import reportRoutes from './routes/reports.js';
import gradeRoutes from './routes/grades.js';
import eventRoutes from './routes/events.js';
import openSessionRoutes from './routes/open-sessions.js';
import supportRoutes from './routes/support.js';
import telegramBotRoutes from './routes/telegram-bot.js';
import ratingRoutes from './routes/ratings.js';
import courseRoutes from './routes/courses.js';
import assistantRoutes from './routes/assistant.js';
import premiumRoutes from './routes/premium.js';
import teacherRoutes from './routes/teacher.js';
import { signatures as signatureRoutes } from './routes/signatures.js';
import { lessonApprovals as lessonApprovalRoutes } from './routes/lesson-approvals.js';
import { structuredLog, nowISO } from '@tarbie/shared';
import type { QueueMessage } from '@tarbie/shared';
import { sendRatingRequest } from './routes/telegram-bot.js';
```

```
const app = new Hono<HonoEnv>();
```

```
app.use('*', corsMiddleware);
app.use('*', bodyLimit());
```

```
app.route('/api/auth', authRoutes);
app.route('/api/sessions', sessionRoutes);
app.route('/api/sessions', attendanceRoutes);
app.route('/api/notifications', notificationRoutes);
app.route('/api/admin', adminRoutes);
app.route('/api/reports', reportRoutes);
app.route('/api/grades', gradeRoutes);
app.route('/api/events', eventRoutes);
app.route('/api/open-sessions', openSessionRoutes);
app.route('/api/support', supportRoutes);
app.route('/api/telegram', telegramBotRoutes);
app.route('/api/ratings', ratingRoutes);
app.route('/api/courses', courseRoutes);
app.route('/api/assistant', assistantRoutes);
app.route('/api/premium', premiumRoutes);
app.route('/api/teacher', teacherRoutes);
app.route('/api/signatures', signatureRoutes);
app.route('/api/lesson-approvals', lessonApprovalRoutes);
```

```
app.get('/api/health', (c) => {
  return c.json({ success: true, data: { status: 'ok', timestamp: new
Date().toISOString() } });
});
```

```
app.notFound((c) => {
  return c.json({ success: false, code: 'NOT_FOUND', message: 'Route not found' }, 404);
});
```

```
app.onError((err, c) => {
```

```

    structuredLog('error', 'Unhandled error', { error: err.message, path: c.req.path });
    return c.json({ success: false, code: 'INTERNAL_ERROR', message: 'Internal server
error' }, 500);
});

// Cloudflare Queues hard-limit message body to 128 KB. With ~36-byte UUIDs,
// 100 user IDs ≈ 3.6 KB plus overhead – safe and parallelisable.
const QUEUE_FANOUT_CHUNK = 100;

async function queueFanOut(
  env: Env,
  base: Omit<QueueMessage, 'user_ids'>,
  userIds: string[]
): Promise<void> {
  if (userIds.length === 0) return;
  for (let i = 0; i < userIds.length; i += QUEUE_FANOUT_CHUNK) {
    const slice = userIds.slice(i, i + QUEUE_FANOUT_CHUNK);
    await env.NOTIFICATION_QUEUE.send({ ...base, user_ids: slice });
  }
}

async function handleCron(event: ScheduledEvent, env: Env, ctx: ExecutionContext):
Promise<void> {
  const cron = event.cron;
  structuredLog('info', 'Cron triggered', { cron });

  if (cron === '0 4 * * *') {
    const tomorrow = new Date();
    tomorrow.setDate(tomorrow.getDate() + 1);
    const tomorrowStr = tomorrow.toISOString().split('T')[0];

    // 1 JOIN query collects sessions + class students for tomorrow (eliminates N+1).
    const reminderRows = await env.DB.prepare(
      `SELECT ts.id as session_id, ts.topic, ts.planned_date, ts.teacher_id,
         c.name as class_name, cs.student_id
      FROM tarbie_sessions ts
      JOIN classes c ON ts.class_id = c.id
      LEFT JOIN class_students cs ON cs.class_id = ts.class_id
      WHERE ts.planned_date = ? AND ts.status = 'planned'`
    ).bind(tomorrowStr).all<{
      session_id: string; topic: string; planned_date: string;
      teacher_id: string; class_name: string; student_id: string | null;
    }>();

    // Group rows by session.
    const sessionMap = new Map<string, {
      topic: string; planned_date: string; teacher_id: string;
      class_name: string; student_ids: Set<string>;
    }>();
    for (const r of reminderRows.results) {
      let s = sessionMap.get(r.session_id);
      if (!s) {
        s = { topic: r.topic, planned_date: r.planned_date, teacher_id: r.teacher_id,
              class_name: r.class_name, student_ids: new Set() };
        sessionMap.set(r.session_id, s);
      }
      if (r.student_id) s.student_ids.add(r.student_id);
    }

    for (const [sessionId, s] of sessionMap) {
      const userIds = [s.teacher_id, ...s.student_ids];
      await queueFanOut(env, {
        event_type: 'SESSION_REMINDER',
        session_id: sessionId,
        template_vars: {
          topic: s.topic,
          date: s.planned_date,

```



```

        class_name: s.class_name,
      },
      attempt: 0,
    }, userIds);
  }

structuredLog('info', 'Daily reminders queued', { count: sessionMap.size });

// — Curator notifications: sessions in next 7 days with empty topic —
const weekAhead = new Date();
weekAhead.setDate(weekAhead.getDate() + 7);
const weekAheadStr = weekAhead.toISOString().split('T')[0];
const todayStr2 = new Date().toISOString().split('T')[0];

const emptyTopicSessions = await env.DB.prepare(
  `SELECT ts.id, ts.topic, ts.planned_date, ts.teacher_id, c.name as class_name
  FROM tarbie_sessions ts
  JOIN classes c ON ts.class_id = c.id
  WHERE ts.planned_date BETWEEN ? AND ?
  AND ts.status = 'planned'
  AND (ts.topic IS NULL OR ts.topic = '')`
).bind(todayStr2, weekAheadStr).all<{
  id: string; topic: string; planned_date: string;
  teacher_id: string; class_name: string;
}>();

for (const session of emptyTopicSessions.results) {
  const reminderMsg: QueueMessage = {
    event_type: 'TOPIC_REMINDER',
    session_id: session.id,
    user_ids: [session.teacher_id],
    template_vars: {
      topic: '',
      date: session.planned_date,
      class_name: session.class_name,
    },
    attempt: 0,
  };
  await env.NOTIFICATION_QUEUE.send(reminderMsg);
}

structuredLog('info', 'Topic reminders queued', { count:
emptyTopicSessions.results.length });
}

// Weekly incomplete check (runs on Fridays inside the daily cron)
if (cron === '0 4 * * *' && new Date().getUTCDay() === 5) {
  const weekStart = new Date();
  weekStart.setDate(weekStart.getDate() - 5);
  const weekStartStr = weekStart.toISOString().split('T')[0];
  const todayStr = new Date().toISOString().split('T')[0];

  const incomplete = await env.DB.prepare(
    `SELECT ts.id, ts.topic, ts.class_id, c.name as class_name, c.school_id
    FROM tarbie_sessions ts
    JOIN classes c ON ts.class_id = c.id
    WHERE ts.planned_date BETWEEN ? AND ? AND ts.status = 'planned'`
  ).bind(weekStartStr, todayStr).all<{
    id: string; topic: string; class_id: string; class_name: string; school_id: string;
  }>();

  structuredLog('info', 'Weekly incomplete check', { count:
incomplete.results.length });
}

// — Auto-complete sessions whose time has passed (runs daily at 09:00 local) —

```

```

if (cron === '0 4 * * *') {
  const yesterday = new Date();
  yesterday.setDate(yesterday.getDate() - 1);
  const yesterdayStr = yesterday.toISOString().split('T')[0]!;

  // Fetch all sessions + their class students + school admins in 2 joined queries (no
  N+1).
  const pastRows = await env.DB.prepare(
    `SELECT ts.id as session_id, ts.topic, ts.planned_date, ts.class_id,
      c.name as class_name, c.school_id, cs.student_id
    FROM tarbie_sessions ts
    JOIN classes c ON ts.class_id = c.id
    LEFT JOIN class_students cs ON cs.class_id = ts.class_id
    WHERE ts.planned_date <= ? AND ts.status = 'planned'`
  ).bind(yesterdayStr).all<{
    session_id: string; topic: string; planned_date: string; class_id: string;
    class_name: string; school_id: string; student_id: string | null;
  }>();

  if (pastRows.results.length === 0) {
    return;
  }

  // Group sessions and collect distinct school_ids for an admin lookup batch.
  const sessions = new Map<string, {
    topic: string; planned_date: string; class_id: string; class_name: string;
    school_id: string; student_ids: Set<string>;
  }>();
  const schoolIds = new Set<string>();
  for (const r of pastRows.results) {
    let s = sessions.get(r.session_id);
    if (!s) {
      s = { topic: r.topic, planned_date: r.planned_date, class_id: r.class_id,
        class_name: r.class_name, school_id: r.school_id, student_ids: new Set() };
      sessions.set(r.session_id, s);
      schoolIds.add(r.school_id);
    }
    if (r.student_id) s.student_ids.add(r.student_id);
  }

  // 1 query for all school admins.
  const adminsBySchool = new Map<string, string[]>();
  if (schoolIds.size > 0) {
    const schoolList = Array.from(schoolIds);
    const ph = schoolList.map(() => '?').join(',');
    const admins = await env.DB.prepare(
      `SELECT id, school_id FROM users WHERE school_id IN (${ph}) AND role = 'admin'`
    ).bind(...schoolList).all<{ id: string; school_id: string }>();
    for (const a of admins.results) {
      const arr = adminsBySchool.get(a.school_id) ?? [];
      arr.push(a.id);
      adminsBySchool.set(a.school_id, arr);
    }
  }

  // Batch UPDATE all sessions to completed.
  const nowStr = nowISO();
  const sessionList = Array.from(sessions.entries());
  const UPDATE_CHUNK = 50;
  for (let i = 0; i < sessionList.length; i += UPDATE_CHUNK) {
    const slice = sessionList.slice(i, i + UPDATE_CHUNK);
    const stmts = slice.map(([sid, s]) =>
      env.DB.prepare(
        `UPDATE tarbie_sessions SET status = 'completed', actual_date = ?, updated_at =
? WHERE id = ?`
      ).bind(s.planned_date, nowStr, sid)
    );
    await env.DB.batch(stmts);
  }

```

```

}

// Queue SESSION_COMPLETED notifications (chunked fan-out).
for (const [sessionId, s] of sessions) {
  const admins = adminsBySchool.get(s.school_id) ?? [];
  const userIds = [...admins, ...s.student_ids];
  await queueFanOut(env, {
    event_type: 'SESSION_COMPLETED',
    session_id: sessionId,
    template_vars: {
      topic: s.topic,
      class_name: s.class_name,
      attendance_count: '0',
      total_students: String(s.student_ids.size),
    },
    attempt: 0,
  }, userIds);
}

// Rating requests run async via waitUntil so cron doesn't block on Telegram HTTP.
// Each call still loops through students sequentially inside; for very large schools
// consider moving this to its own queue event in the future.
ctx.waitUntil(async () => {
  for (const [sessionId, s] of sessions) {
    try {
      await sendRatingRequest(sessionId, s.topic, env, env.TELEGRAM_BOT_TOKEN);
    } catch (err) {
      structuredLog('warn', 'sendRatingRequest failed', {
        session_id: sessionId,
        error: err instanceof Error ? err.message : 'unknown',
      });
    }
  }
})());

structuredLog('info', 'Auto-completed past sessions', { count: sessions.size });
}

export default {
  fetch: app.fetch,
  scheduled: handleCron,
};

```

Электронды қолтаңба модулі

Қолтаңба модулі пайдаланушының электронды қолтаңбасын сақтайды және құжатты бекіту процесінде пайдаланылады.

routes/signatures.ts

```
import { Hono } from 'hono';
import type { HonoEnv } from '../env.js';
import { authMiddleware, requireRole } from '../middleware/auth.js';
import { generateId, ERROR_CODES } from '@tarbie/shared';

const signatures = new Hono<HonoEnv>();

signatures.use('*', authMiddleware);

// — Get my signature —
signatures.get('/me', async (c) => {
  const user = c.get('user');
  const sig = await c.env.DB.prepare(
    'SELECT id, signature_data, signature_data_2, created_at, updated_at FROM user_signatures WHERE user_id = ?'
  ).bind(user.id).first<{ id: string; signature_data: string; signature_data_2: string | null; created_at: string; updated_at: string }>();

  return c.json({ success: true, data: sig || null });
});

// — Save/update my signature —
signatures.post('/me', async (c) => {
  const user = c.get('user');
  const body = await c.req.json() as { signature_data: string; signature_data_2?: string };

  if (!body.signature_data || typeof body.signature_data !== 'string') {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'signature_data is required' }, 400);
  }

  // Only curators (teachers) and admins can have signatures
  if (user.role === 'student') {
    return c.json({ success: false, code: ERROR_CODES.FORBIDDEN, message: 'Students cannot set signatures' }, 403);
  }

  const sig2 = (user.role === 'admin' && body.signature_data_2) ? body.signature_data_2 : null;

  const now = new Date().toISOString();
  const existing = await c.env.DB.prepare(
    'SELECT id FROM user_signatures WHERE user_id = ?'
  ).bind(user.id).first<{ id: string }>();

  if (existing) {
    await c.env.DB.prepare(
      'UPDATE user_signatures SET signature_data = ?, signature_data_2 = ?, updated_at = ? WHERE user_id = ?'
    ).bind(body.signature_data, sig2, now, user.id).run();
    return c.json({ success: true, data: { id: existing.id, updated: true } });
  } else {
    const id = generateId();
    await c.env.DB.prepare(
      'INSERT INTO user_signatures (id, user_id, signature_data, signature_data_2, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?)'
    ).bind(id, user.id, body.signature_data, sig2, now, now).run();
  }
});
```

```

    ).bind(id, user.id, body.signature_data, sig2, now, now).run();
    return c.json({ success: true, data: { id, updated: false } }, 201);
  }
});

// — Check if user has signature (for first-login prompt) —
signatures.get('/check', async (c) => {
  const user = c.get('user');

  // Students don't need signatures
  if (user.role === 'student') {
    return c.json({ success: true, data: { has_signature: true, required: false } });
  }

  const sig = await c.env.DB.prepare(
    'SELECT id FROM user_signatures WHERE user_id = ?'
  ).bind(user.id).first<{ id: string }>();

  return c.json({ success: true, data: { has_signature: !!sig, required: true } });
});

// — Admin: get any user's signature —
signatures.get('/users/:userId', requireRole('admin'), async (c) => {
  const userId = c.req.param('userId');
  const sig = await c.env.DB.prepare(
    'SELECT id, user_id, signature_data, created_at, updated_at FROM user_signatures
WHERE user_id = ?'
  ).bind(userId).first<{ id: string; user_id: string; signature_data: string; created_at:
string; updated_at: string }>();

  if (!sig) {
    return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Signature
not found' }, 404);
  }

  return c.json({ success: true, data: sig });
});

export { signatures };

```

Клиенттік API қабаты

Клиенттік бөліктегі ApiClient серверге сұраныстарды жібереді, токенді автоматты түрде Қосады және жауаптар мен қателерді біркелкі өңдейді.

lib/api.ts

```
import type { ApiResponse, ApiError } from '@tarbie/shared';

const API_BASE = import.meta.env.VITE_API_URL ??
'https://dprabota.bahtyarsanzhar.workers.dev';

class ApiClient {
  private token: string | null = null;
  private onUnauthorized: (() => void) | null = null;

  setToken(token: string | null): void {
    this.token = token;
  }

  setOnUnauthorized(cb: () => void): void {
    this.onUnauthorized = cb;
  }

  private async request<T>(
    method: string,
    path: string,
    body?: unknown,
    params?: Record<string, string>
  ): Promise<T> {
    let url = `${API_BASE}${path}`;

    if (params) {
      const searchParams = new URLSearchParams();
      Object.entries(params).forEach(([k, v]) => {
        if (v) searchParams.set(k, v);
      });
      const queryString = searchParams.toString();
      if (queryString) {
        url += `?${queryString}`;
      }
    }

    const headers: Record<string, string> = {};

    if (body !== undefined) {
      headers['Content-Type'] = 'application/json';
    }

    if (this.token) {
      headers['Authorization'] = `Bearer ${this.token}`;
    }

    const res = await fetch(url, {
      method,
      headers,
      body: body ? JSON.stringify(body) : undefined,
    });

    const contentType = res.headers.get('content-type');
    if (!contentType || !contentType.includes('application/json')) {
      const text = await res.text();
      throw new ApiRequestError('INVALID_RESPONSE', `Expected JSON response but got: ${
        text.slice(0, 100)}`, res.status);
    }
  }
}
```

```

    }

    const text = await res.text();
    if (!text || text.trim() === '') {
        throw new ApiRequestError('EMPTY_RESPONSE', 'Server returned empty response',
res.status);
    }

    let json;
    try {
        json = JSON.parse(text);
    } catch (e) {
        throw new ApiRequestError('INVALID_JSON', `Failed to parse JSON: ${text.slice(0,
100)}`, res.status);
    }

    if (!res.ok) {
        if (res.status === 401 && this.onUnauthorized) {
            this.onUnauthorized();
        }
        const error = json as ApiError;
        throw new ApiRequestError(error.code ?? 'UNKNOWN', error.message ?? 'Request
failed', res.status);
    }

    return (json as ApiResponse<T>).data;
}

private async requestRaw<T>(
    method: string,
    path: string,
    body?: unknown,
    params?: Record<string, string>
): Promise<T> {
    let url = `${API_BASE}${path}`;

    if (params) {
        const searchParams = new URLSearchParams();
        Object.entries(params).forEach(([k, v]) => {
            if (v) searchParams.set(k, v);
        });
        const queryString = searchParams.toString();
        if (queryString) url += `?${queryString}`;
    }

    const headers: Record<string, string> = {};
    if (body !== undefined) headers['Content-Type'] = 'application/json';
    if (this.token) headers['Authorization'] = `Bearer ${this.token}`;

    const res = await fetch(url, {
        method,
        headers,
        body: body ? JSON.stringify(body) : undefined,
    });

    const text = await res.text();
    if (!text || text.trim() === '') {
        throw new ApiRequestError('EMPTY_RESPONSE', 'Server returned empty response',
res.status);
    }

    let json;
    try { json = JSON.parse(text); } catch {
        throw new ApiRequestError('INVALID_JSON', `Failed to parse JSON: ${text.slice(0,
100)}`, res.status);
    }

```

```

    }

    if (!res.ok) {
      const error = json as ApiError;
      throw new ApiRequestError(error.code ?? 'UNKNOWN', error.message ?? 'Request
failed', res.status);
    }

    return json as T;
  }

  async get<T>(path: string, params?: Record<string, string>): Promise<T> {
    return this.request<T>('GET', path, undefined, params);
  }

  async getRaw<T>(path: string, params?: Record<string, string>): Promise<T> {
    return this.requestRaw<T>('GET', path, undefined, params);
  }

  async post<T>(path: string, body?: unknown): Promise<T> {
    return this.request<T>('POST', path, body);
  }

  async put<T>(path: string, body?: unknown): Promise<T> {
    return this.request<T>('PUT', path, body);
  }

  async patch<T>(path: string, body?: unknown): Promise<T> {
    return this.request<T>('PATCH', path, body);
  }

  async delete<T>(path: string): Promise<T> {
    return this.request<T>('DELETE', path);
  }

  async postFormData<T>(path: string, formData: FormData): Promise<T> {
    const url = `${API_BASE}${path}`;
    const headers: Record<string, string> = {};
    if (this.token) {
      headers['Authorization'] = `Bearer ${this.token}`;
    }

    const res = await fetch(url, {
      method: 'POST',
      headers,
      body: formData,
    });

    const text = await res.text();
    if (!text || text.trim() === '') {
      throw new ApiRequestError('EMPTY_RESPONSE', 'Server returned empty response',
res.status);
    }

    let json;
    try { json = JSON.parse(text); } catch {
      throw new ApiRequestError('INVALID_JSON', `Failed to parse JSON: ${text.slice(0,
100)}`, res.status);
    }

    if (!res.ok) {
      if (res.status === 401 && this.onUnauthorized) {
        this.onUnauthorized();
      }
    }
  }

```



```

    }
    const error = json as ApiError;
    throw new ApiRequestError(error.code ?? 'UNKNOWN', error.message ?? 'Request
failed', res.status);
  }

  return (json as ApiResponse<T>).data;
}

export class ApiRequestError extends Error {
  constructor(
    public code: string,
    message: string,
    public status: number
  ) {
    super(message);
    this.name = 'ApiRequestError';
  }
}

export const api = new ApiClient();

```

Деректерді валидациялау схемалары (Zod)

Zod схемалары клиент пен серверде ортақ пайдаланылып, кіріс деректердің дұрыстығын тексереді және типтердің сәйкестігін қамтамасыз етеді.

shared/schemas.ts

```
import { z } from 'zod';

const phoneRegex = /^+7\d{10}$/;

export const langSchema = z.enum(['kz', 'ru']);
export const roleSchema = z.enum(['admin', 'teacher', 'student']);
export const sessionStatusSchema = z.enum(['pending_approval', 'planned', 'completed', 'cancelled', 'rescheduled']);
export const attendanceStatusSchema = z.enum(['present', 'absent', 'late', 'excused']);
export const gradeStatusSchema = z.enum(['present', 'absent', 'makeup']);
export const notificationEventTypeSchema = z.enum([
  'SESSION_PLANNED',
  'SESSION_REMINDER',
  'SESSION_COMPLETED',
  'SESSION_RESCHEDULED',
  'ABSENCE_ALERT',
  'TOPIC_REMINDER',
]);

export const loginSchema = z.object({
  phone: z.string().regex(phoneRegex, 'Phone must be in +7XXXXXXXXXX format'),
});

export const verifyOtpSchema = z.object({
  phone: z.string().regex(phoneRegex, 'Phone must be in +7XXXXXXXXXX format'),
  otp: z.string().length(6, 'OTP must be 6 digits').regex(/^\d{6}$/, 'OTP must contain only digits'),
});

export const createSessionSchema = z.object({
  class_id: z.string().uuid(),
  topic: z.string().min(1).max(500),
  planned_date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Date must be YYYY-MM-DD'),
  time_slot: z.string().regex(/^\d{2}:\d{2}$/, 'Time slot must be HH:MM'),
  room: z.string().min(1).max(50),
  duration_minutes: z.number().int().min(15).max(180).default(30),
  notes: z.string().max(2000).nullable().optional(),
});

export const updateSessionSchema = z.object({
  topic: z.string().min(1).max(500).optional(),
  planned_date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Date must be YYYY-MM-DD').optional(),
  time_slot: z.string().regex(/^\d{2}:\d{2}$/, 'Time slot must be HH:MM').optional(),
  room: z.string().min(1).max(50).optional(),
  duration_minutes: z.number().int().min(15).max(180).optional(),
  notes: z.string().max(2000).nullable().optional(),
  status: sessionStatusSchema.optional(),
});

export const completeSessionSchema = z.object({
  actual_date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Date must be YYYY-MM-DD').optional(),
  notes: z.string().max(2000).nullable().optional(),
  attachment_url: z.string().url().nullable().optional(),
});
```

```

export const attendanceEntrySchema = z.object({
  student_id: z.string().uuid(),
  status: attendanceStatusSchema,
});

export const bulkAttendanceSchema = z.object({
  attendance: z.array(attendanceEntrySchema).min(1),
});

export const createUserSchema = z.object({
  full_name: z.string().min(2).max(200),
  role: roleSchema,
  phone: z.string().regex(phoneRegex, 'Phone must be in +7XXXXXXXXXX format'),
  lang: langSchema.default('ru'),
  telegram_chat_id: z.string().nullable().optional(),
});

export const.createClassSchema = z.object({
  name: z.string().min(1).max(20),
  teacher_id: z.string().uuid(),
  academic_year: z.string().regex(/^\d{4}-\d{4}$/, 'Must be YYYY-YYYY format'),
});

export const sessionsQuerySchema = z.object({
  classId: z.string().uuid().optional(),
  month: z.string().regex(/^\d{4}-\d{2}$/, 'Must be YYYY-MM format').optional(),
  status: sessionStatusSchema.optional(),
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().min(1).max(2500).default(50),
});

export const monthlyReportQuerySchema = z.object({
  classId: z.string().uuid(),
  month: z.string().regex(/^\d{4}-\d{2}$/, 'Must be YYYY-MM format'),
});

export const sendNotificationSchema = z.object({
  session_id: z.string().uuid(),
  event_type: notificationEventTypeSchema,
  user_ids: z.array(z.string().uuid()).min(1).optional(),
});

export const notificationLogQuerySchema = z.object({
  userId: z.string().uuid().optional(),
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().min(1).max(100).default(20),
});

export const updateNotificationTemplateSchema = z.object({
  template_text: z.string().min(1).max(2000),
});

export const schoolSchema = z.object({
  name: z.string().min(2).max(300),
  bin: z.string().length(12, 'BIN must be 12 digits').regex(/^\d{12}$/, 'BIN must contain only digits'),
  city: z.string().min(2).max(100),
});

export const gradeEntrySchema = z.object({
  student_id: z.string().uuid(),
  status: gradeStatusSchema,
  grade: z.number().int().min(0).max(100).nullable(),
  comment: z.string().max(500).nullable().optional(),
});

```

```

export const bulkGradeSchema = z.object({
  grades: z.array(gradeEntrySchema).min(1),
});

export const courseStatusSchema = z.enum(['draft', 'published', 'archived']);
export const lessonTypeSchema = z.enum(['video', 'text']);
export const enrollmentStatusSchema = z.enum(['active', 'completed', 'cancelled']);
export const lessonProgressStatusSchema = z.enum(['not_started', 'in_progress', 'completed']);

export const createCourseSchema = z.object({
  title: z.string().min(1).max(500),
  description: z.string().max(5000).default(''),
  category_id: z.string().nullable().optional(),
  price: z.number().min(0).default(0),
  cover_url: z.string().url().nullable().optional(),
  lang: langSchema.default('ru'),
});

export const updateCourseSchema = z.object({
  title: z.string().min(1).max(500).optional(),
  description: z.string().max(5000).optional(),
  category_id: z.string().nullable().optional(),
  price: z.number().min(0).optional(),
  status: courseStatusSchema.optional(),
  cover_url: z.string().url().nullable().optional(),
  lang: langSchema.optional(),
});

export const createModuleSchema = z.object({
  title: z.string().min(1).max(300),
  sort_order: z.number().int().min(0).default(0),
});

export const updateModuleSchema = z.object({
  title: z.string().min(1).max(300).optional(),
  sort_order: z.number().int().min(0).optional(),
});

export const createLessonSchema = z.object({
  title: z.string().min(1).max(500),
  type: lessonTypeSchema.default('text'),
  content: z.string().max(50000).default(''),
  video_url: z.string().url().nullable().optional(),
  duration_minutes: z.number().int().min(0).default(0),
  sort_order: z.number().int().min(0).default(0),
});

export const updateLessonSchema = z.object({
  title: z.string().min(1).max(500).optional(),
  type: lessonTypeSchema.optional(),
  content: z.string().max(50000).optional(),
  video_url: z.string().url().nullable().optional(),
  duration_minutes: z.number().int().min(0).optional(),
  sort_order: z.number().int().min(0).optional(),
});

export const coursesQuerySchema = z.object({
  categoryId: z.string().optional(),
  status: courseStatusSchema.optional(),
  search: z.string().optional(),
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().min(1).max(100).default(20),
});

```

```
export const createReviewSchema = z.object({
  rating: z.number().int().min(1).max(5),
  text: z.string().max(2000).default(''),
});

export const updateProgressSchema = z.object({
  status: lessonProgressStatusSchema,
});
```

Тәрбие сағаттарының бизнес-логикасы

sessions.ts модулі тәрбие сағаттарын құру, өңдеу, тағайындау, аяқтау және жою операцияларын, сондай-ақ рөлдік шектеулерді жүзеге асырады.

routes/sessions.ts

```
import { Hono } from 'hono';
import type { HonoEnv } from '../env.js';
import { authMiddleware, requireRole } from '../middleware/auth.js';
import {
  createSessionSchema, updateSessionSchema, completeSessionSchema,
  sessionsQuerySchema, generateId, nowISO, ERROR_CODES, structuredLog,
  isValidRoom, isValidTimeSlot,
} from '@tarbie/shared';
import type { QueueMessage } from '@tarbie/shared';
import { sendRatingRequest } from '../telegram-bot.js';

const sessions = new Hono<HonoEnv>();

sessions.use('*', authMiddleware);

// Get classes accessible to current user (for dropdowns etc.)
sessions.get('/classes', async (c) => {
  const user = c.get('user');
  let rows;
  if (user.role === 'admin') {
    rows = await c.env.DB.prepare(
      `SELECT cl.id, cl.name, cl.teacher_id, cl.academic_year, u.full_name as
teacher_name,
      (SELECT COUNT(*) FROM class_students WHERE class_id = cl.id) as student_count
      FROM classes cl JOIN users u ON cl.teacher_id = u.id
      WHERE cl.school_id = ? ORDER BY cl.name`
    ).bind(user.school_id).all();
  } else if (user.role === 'teacher') {
    rows = await c.env.DB.prepare(
      `SELECT cl.id, cl.name, cl.teacher_id, cl.academic_year, u.full_name as
teacher_name,
      (SELECT COUNT(*) FROM class_students WHERE class_id = cl.id) as student_count
      FROM classes cl JOIN users u ON cl.teacher_id = u.id
      WHERE cl.teacher_id = ? ORDER BY cl.name`
    ).bind(user.id).all();
  } else {
    rows = await c.env.DB.prepare(
      `SELECT cl.id, cl.name, cl.teacher_id, cl.academic_year, u.full_name as
teacher_name,
      (SELECT COUNT(*) FROM class_students WHERE class_id = cl.id) as student_count
      FROM classes cl JOIN users u ON cl.teacher_id = u.id
      JOIN class_students cs ON cs.class_id = cl.id
      WHERE cs.student_id = ? ORDER BY cl.name`
    ).bind(user.id).all();
  }
  return c.json({ success: true, data: rows.results });
});

sessions.get('/', async (c) => {
  const user = c.get('user');
  const query = sessionsQuerySchema.safeParse(Object.fromEntries(new
URL(c.req.url).searchParams));
  if (!query.success) {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
query.error.issues[0]?.message ?? 'Invalid query' }, 400);
  }

  const { classId, month, status, page, pageSize } = query.data;
  const conditions: string[] = [];
  const params: string[] = [];
```

```

    if (user.role === 'teacher') {
      conditions.push('ts.teacher_id = ?');
      params.push(user.id);
    } else if (user.role === 'student') {
      conditions.push('ts.class_id IN (SELECT class_id FROM class_students WHERE student_id
= ?)');
      params.push(user.id);
      // Students should not see sessions pending approval
      conditions.push("ts.status != 'pending_approval'");
    } else {
      conditions.push('c.school_id = ?');
      params.push(user.school_id);
    }
  }

  if (classId) {
    conditions.push('ts.class_id = ?');
    params.push(classId);
  }
  if (month) {
    conditions.push("ts.planned_date LIKE ? || '%"");
    params.push(month);
  }
  if (status) {
    conditions.push('ts.status = ?');
    params.push(status);
  }
}

const where = conditions.length > 0 ? `WHERE ${conditions.join(' AND ')} ` : '';
const offset = (page - 1) * pageSize;

const countResult = await c.env.DB.prepare(
  `SELECT COUNT(*) as total FROM tarbie_sessions ts JOIN classes c ON ts.class_id =
c.id ${where}`
).bind(...params).first<{ total: number }>();

const rows = await c.env.DB.prepare(
  `SELECT ts.*, c.name as class_name, u.full_name as teacher_name,
    u.avatar_url as teacher_avatar_url
  FROM tarbie_sessions ts
  JOIN classes c ON ts.class_id = c.id
  JOIN users u ON ts.teacher_id = u.id
  ${where}
  ORDER BY ts.planned_date DESC
  LIMIT ? OFFSET ?`
).bind(...params, pageSize, offset).all();

return c.json({
  success: true,
  data: rows.results,
  total: countResult?.total ?? 0,
  page,
  pageSize,
});
});

sessions.post('/', requireRole('admin', 'teacher'), async (c) => {
  const user = c.get('user');
  const body = await c.req.json();
  const parsed = createSessionSchema.safeParse(body);
  if (!parsed.success) {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
parsed.error.issues[0]?.message ?? 'Invalid input' }, 400);
  }
}

```

```

    const { class_id, topic, planned_date, time_slot, room, duration_minutes, notes } =
    parsed.data;

    // Validate time slot and room
    if (!isValidTimeSlot(time_slot)) {
        return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Invalid
time slot. Must be a valid 30-min college slot.' }, 400);
    }
    if (!isValidRoom(room)) {
        return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Invalid
room. Choose a valid college room.' }, 400);
    }

    const cls = await c.env.DB.prepare('SELECT id, teacher_id, name FROM classes WHERE id =
? AND school_id = ?')
        .bind(class_id, user.school_id).first<{ id: string; teacher_id: string; name:
string }>();
    if (!cls) {
        return c.json({ success: false, code: ERROR_CODES.CLASS_NOT_FOUND, message: 'Class
not found' }, 404);
    }

    // Teacher can only create sessions for their own classes
    if (user.role === 'teacher' && cls.teacher_id !== user.id) {
        return c.json({ success: false, code: 'FORBIDDEN', message: 'You can only create
sessions for your own classes' }, 403);
    }

    // Room conflict check: same date + same time_slot + same room + not cancelled
    const conflict = await c.env.DB.prepare(
        `SELECT id, topic FROM tarbie_sessions WHERE planned_date = ? AND time_slot = ? AND
room = ? AND status != 'cancelled'`
    ).bind(planned_date, time_slot, room).first<{ id: string; topic: string }>();
    if (conflict) {
        return c.json({ success: false, code: 'ROOM_CONFLICT', message: `Room ${room} is
already booked at ${time_slot} on ${planned_date}` }, 409);
    }

    const teacherId = user.role === 'teacher' ? user.id : cls.teacher_id;
    const id = generateId();
    const now = nowISO();

    // Curators need admin approval; admins create directly as planned
    const initialStatus = user.role === 'teacher' ? 'pending_approval' : 'planned';

    await c.env.DB.prepare(
        `INSERT INTO tarbie_sessions (id, class_id, teacher_id, topic, planned_date,
time_slot, room, status, duration_minutes, notes, created_at, updated_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`
    ).bind(id, class_id, teacherId, topic, planned_date, time_slot, room, initialStatus,
duration_minutes, notes ?? null, now, now).run();

    // Only send SESSION_PLANNED notification if admin created it directly
    if (initialStatus === 'planned') {
        const students = await c.env.DB.prepare(
            'SELECT student_id FROM class_students WHERE class_id = ?'
        ).bind(class_id).all<{ student_id: string }>();

        const userIds = [teacherId, ...students.results.map(s => s.student_id)];

        const queueMsg: QueueMessage = {
            event_type: 'SESSION_PLANNED',
            session_id: id,
            user_ids: userIds,

```



```

        template_vars: { topic, date: planned_date, class_name: cls.name, teacher_name:
    '' },
        attempt: 0,
    };
    await c.env.NOTIFICATION_QUEUE.send(queueMsg);
}

    structuredLog('info', 'Session created', { session_id: id, class_id, teacher_id:
teacherId, status: initialStatus });

    return c.json({ success: true, data: { id, class_id, teacher_id: teacherId, topic,
planned_date, time_slot, room, status: initialStatus, duration_minutes } }, 201);
});

sessions.put('/:id', requireRole('admin', 'teacher'), async (c) => {
    const user = c.get('user');
    const sessionId = c.req.param('id');
    const body = await c.req.json();
    const parsed = updateSessionSchema.safeParse(body);
    if (!parsed.success) {
        return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
parsed.error.issues[0]?.message ?? 'Invalid input' }, 400);
    }

    const existing = await c.env.DB.prepare(
        `SELECT ts.*, c.name as class_name, c.school_id FROM tarbie_sessions ts JOIN classes
c ON ts.class_id = c.id WHERE ts.id = ?`
    ).bind(sessionId).first<{ id: string; class_id: string; teacher_id: string; topic:
string; planned_date: string; status: string; class_name: string; school_id: string }>();

    if (!existing || existing.school_id !== user.school_id) {
        return c.json({ success: false, code: ERROR_CODES.SESSION_NOT_FOUND, message:
'Session not found' }, 404);
    }

    // Teacher can only update their own sessions
    if (user.role === 'teacher' && existing.teacher_id !== user.id) {
        return c.json({ success: false, code: 'FORBIDDEN', message: 'You can only edit your
own sessions' }, 403);
    }

    const updates: string[] = [];
    const values: (string | number)[] = [];
    const data = parsed.data;

    if (data.topic !== undefined) { updates.push('topic = ?'); values.push(data.topic); }
    if (data.planned_date !== undefined) { updates.push('planned_date = ?');
values.push(data.planned_date); }
    if (data.time_slot !== undefined) {
        if (!isValidTimeSlot(data.time_slot)) {
            return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
'Invalid time slot' }, 400);
        }
        updates.push('time_slot = ?'); values.push(data.time_slot);
    }
    if (data.room !== undefined) {
        if (!isValidRoom(data.room)) {
            return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
'Invalid room' }, 400);
        }
        updates.push('room = ?'); values.push(data.room);
    }
    if (data.duration_minutes !== undefined) { updates.push('duration_minutes = ?');
values.push(data.duration_minutes); }
    if (data.notes !== undefined) { updates.push('notes = ?'); values.push(data.notes ??
''); }

```

```

    if (data.status !== undefined) { updates.push('status = ?');
    values.push(data.status); }

    // Room conflict check if date/time/room changed
    const newDate = data.planned_date ?? existing.planned_date;
    const newSlot = data.time_slot ?? (existing as unknown as { time_slot: string |
    null }).time_slot;
    const newRoom = data.room ?? (existing as unknown as { room: string | null }).room;
    if (newSlot && newRoom) {
        const conflict = await c.env.DB.prepare(
            `SELECT id FROM tarbie_sessions WHERE planned_date = ? AND time_slot = ? AND room =
            ? AND status != 'cancelled' AND id != ?`
        ).bind(newDate, newSlot, newRoom, sessionId).first<{ id: string }>();
        if (conflict) {
            return c.json({ success: false, code: 'ROOM_CONFLICT', message: `Room ${newRoom} is
            already booked at ${newSlot} on ${newDate}` }, 409);
        }
    }

    updates.push('updated_at = ?');
    values.push(nowISO());
    values.push(sessionId);

    await c.env.DB.prepare(
        `UPDATE tarbie_sessions SET ${updates.join(', ')} WHERE id = ?`
    ).bind(...values).run();

    const isRescheduled = data.planned_date && data.planned_date !== existing.planned_date;
    if (isRescheduled) {
        const students = await c.env.DB.prepare(
            `SELECT student_id FROM class_students WHERE class_id = ?`
        ).bind(existing.class_id).all<{ student_id: string }>();

        const queueMsg: QueueMessage = {
            event_type: 'SESSION_RESCHEDULED',
            session_id: sessionId,
            user_ids: [existing.teacher_id, ...students.results.map(s => s.student_id)],
            template_vars: {
                topic: data.topic ?? existing.topic,
                old_date: existing.planned_date,
                new_date: data.planned_date!,
                class_name: existing.class_name,
            },
            attempt: 0,
        };
        await c.env.NOTIFICATION_QUEUE.send(queueMsg);
    }

    structuredLog('info', 'Session updated', { session_id: sessionId, rescheduled: !!
    isRescheduled });

    return c.json({ success: true, data: { id: sessionId, updated: true } });
});

sessions.patch('/:id/complete', requireRole('admin', 'teacher'), async (c) => {
    const user = c.get('user');
    const sessionId = c.req.param('id');
    let body = {};
    try { body = await c.req.json(); } catch { /* empty body is ok */ }
    const parsed = completeSessionSchema.safeParse(body);
    if (!parsed.success) {
        return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message:
        parsed.error.issues[0]?.message ?? 'Invalid input' }, 400);
    }

    const existing = await c.env.DB.prepare(

```

```

        `SELECT ts.*, c.name as class_name, c.school_id FROM tarbie_sessions ts JOIN classes
c ON ts.class_id = c.id WHERE ts.id = ?`
    ).bind(sessionId).first<{ id: string; class_id: string; teacher_id: string; topic:
string; class_name: string; school_id: string }>();

    if (!existing || existing.school_id !== user.school_id) {
        return c.json({ success: false, code: ERROR_CODES.SESSION_NOT_FOUND, message:
'Session not found' }, 404);
    }

    // Teacher can only complete their own sessions
    if (user.role === 'teacher' && existing.teacher_id !== user.id) {
        return c.json({ success: false, code: 'FORBIDDEN', message: 'You can only complete
your own sessions' }, 403);
    }

    const now = nowISO();
    const actualDate = parsed.data.actual_date ?? now.split('T')[0];

    await c.env.DB.prepare(
        `UPDATE tarbie_sessions SET status = 'completed', actual_date = ?, notes =
COALESCE(?, notes), attachment_url = COALESCE(?, attachment_url), updated_at = ? WHERE id
= ?`
    ).bind(actualDate, parsed.data.notes ?? null, parsed.data.attachment_url ?? null, now,
sessionId).run();

    const attendanceCount = await c.env.DB.prepare(
        "SELECT COUNT(*) as cnt FROM session_attendance WHERE session_id = ? AND status IN
('present', 'late')"
    ).bind(sessionId).first<{ cnt: number }>();

    const totalStudents = await c.env.DB.prepare(
        'SELECT COUNT(*) as cnt FROM class_students WHERE class_id = ?'
    ).bind(existing.class_id).first<{ cnt: number }>();

    const admins = await c.env.DB.prepare(
        "SELECT id FROM users WHERE school_id = ? AND role = 'admin'"
    ).bind(user.school_id).all<{ id: string }>();

    const classStudents = await c.env.DB.prepare(
        'SELECT student_id FROM class_students WHERE class_id = ?'
    ).bind(existing.class_id).all<{ student_id: string }>();

    const queueMsg: QueueMessage = {
        event_type: 'SESSION_COMPLETED',
        session_id: sessionId,
        user_ids: [...admins.results.map(a => a.id), ...classStudents.results.map(s =>
s.student_id)],
        template_vars: {
            topic: existing.topic,
            class_name: existing.class_name,
            attendance_count: String(attendanceCount?.cnt ?? 0),
            total_students: String(totalStudents?.cnt ?? 0),
        },
        attempt: 0,
    };
    await c.env.NOTIFICATION_QUEUE.send(queueMsg);

    // Send rating requests to students via Telegram (non-blocking)
    c.executionCtx.waitUntil(
        sendRatingRequest(sessionId, existing.topic, c.env, c.env.TELEGRAM_BOT_TOKEN)
    );

    structuredLog('info', 'Session completed', { session_id: sessionId });

```

```

    return c.json({ success: true, data: { id: sessionId, status: 'completed' } });
  });

sessions.delete('/:id', requireRole('admin', 'teacher'), async (c) => {
  const user = c.get('user');
  const sessionId = c.req.param('id');

  const existing = await c.env.DB.prepare(
    `SELECT ts.id, ts.teacher_id, c.school_id FROM tarbie_sessions ts JOIN classes c ON
    ts.class_id = c.id WHERE ts.id = ?`
  ).bind(sessionId).first<{ id: string; teacher_id: string; school_id: string }>();

  if (!existing || existing.school_id !== user.school_id) {
    return c.json({ success: false, code: ERROR_CODES.SESSION_NOT_FOUND, message:
'Session not found' }, 404);
  }

  // Teacher can only delete their own sessions
  if (user.role === 'teacher' && existing.teacher_id !== user.id) {
    return c.json({ success: false, code: 'FORBIDDEN', message: 'You can only delete your
own sessions' }, 403);
  }

  await c.env.DB.prepare('DELETE FROM tarbie_sessions WHERE id
= ?').bind(sessionId).run();

  structuredLog('info', 'Session deleted', { session_id: sessionId });

  return c.json({ success: true, data: { id: sessionId, deleted: true } });
});

// Get booked rooms for a given date – returns list of { time_slot, room } that are
occupied
sessions.get('/booked-rooms', async (c) => {
  const user = c.get('user');
  const date = c.req.query('date');
  if (!date) {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'date
query param required' }, 400);
  }

  const rows = await c.env.DB.prepare(
    `SELECT ts.time_slot, ts.room, ts.topic, c.name as class_name
    FROM tarbie_sessions ts
    JOIN classes c ON ts.class_id = c.id
    WHERE ts.planned_date = ? AND ts.status != 'cancelled' AND ts.room IS NOT NULL AND
    ts.time_slot IS NOT NULL
    AND c.school_id = ?`
  ).bind(date, user.school_id).all<{ time_slot: string; room: string; topic: string;
class_name: string }>();

  return c.json({ success: true, data: rows.results });
});

export default sessions;

```

Сабақты бекіту және құжатпен жұмыс модулі

lesson-approvals.ts модулі Word-құжатты бекітуге жіберу, Әкімшінің бекітуі немесе қайтаруы және бекіту тарихын сақтау логикасын қамтиды.

routes/lesson-approvals.ts

```
import { Hono } from 'hono';
import type { HonoEnv } from '../env.js';
import { authMiddleware, requireRole } from '../middleware/auth.js';
import { generateId, ERROR_CODES } from '@tarbie/shared';
import type { QueueMessage } from '@tarbie/shared';
import { notifyAdminLessonApproval, notifyCuratorApprovalResult } from '../telegram-bot.js';

const lessonApprovals = new Hono<HonoEnv>();

lessonApprovals.use('*', authMiddleware);

// — Curator: submit lesson for approval (upload Word file) —
lessonApprovals.post('/', requireRole('teacher'), async (c) => {
  const user = c.get('user');
  const formData = await c.req.formData();
  const sessionId = formData.get('session_id') as string;
  const file = formData.get('file') as File | null;

  if (!sessionId || !file) {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'session_id and file are required' }, 400);
  }

  // Validate file type
  const validTypes = [
    'application/msword',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  ];
  if (!validTypes.includes(file.type) && !file.name.endsWith('.doc') && !file.name.endsWith('.docx')) {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Only .doc/.docx files allowed' }, 400);
  }

  // Verify session belongs to this curator
  const session = await c.env.DB.prepare(
    'SELECT id FROM tarbie_sessions WHERE id = ? AND teacher_id = ?'
  ).bind(sessionId, user.id).first<{ id: string }>();

  if (!session) {
    return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Session not found or not yours' }, 404);
  }

  // Check if already submitted
  const existing = await c.env.DB.prepare(
    'SELECT id, status FROM lesson_approvals WHERE session_id = ? AND curator_id = ?'
  ).bind(sessionId, user.id).first<{ id: string; status: string }>();

  if (existing && existing.status === 'pending') {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Already pending approval' }, 400);
  }

  // Upload file to KV as base64 string
  const fileKey = `lesson-approvals:${generateId()}_${file.name}`;
```

```

const arrayBuffer = await file.arrayBuffer();
const uint8 = new Uint8Array(arrayBuffer);
let binaryStr = '';
for (let i = 0; i < uint8.length; i++) {
  binaryStr += String.fromCharCode(uint8[i]);
}
await c.env.KV.put(fileKey, btoa(binaryStr), { metadata: { contentType: file.type,
fileName: file.name } });

const now = new Date().toISOString();
const id = generateId();

// Get curator's signature
const curatorSig = await c.env.DB.prepare(
  'SELECT id FROM user_signatures WHERE user_id = ?'
).bind(user.id).first<{ id: string }>();

await c.env.DB.prepare(
  `INSERT INTO lesson_approvals (id, session_id, curator_id, school_id, word_file_url,
word_file_name, status, curator_signature_id, created_at, updated_at)
  VALUES (?, ?, ?, ?, ?, ?, 'pending', ?, ?, ?)`
).bind(id, sessionId, user.id, user.school_id, fileKey, file.name, curatorSig?.id ||
null, now, now).run();

// Notify admin via Telegram
const session2 = await c.env.DB.prepare(
  'SELECT topic FROM tarbie_sessions WHERE id = ?'
).bind(sessionId).first<{ topic: string }>();
const curatorUser = await c.env.DB.prepare(
  'SELECT full_name FROM users WHERE id = ?'
).bind(user.id).first<{ full_name: string }>();
c.executionCtx.waitUntil(
  notifyAdminLessonApproval(curatorUser?.full_name ?? '', session2?.topic ?? 'Без
темы', id, c.env, c.env.TELEGRAM_BOT_TOKEN)
  .catch(() => {})
);

return c.json({ success: true, data: { id, status: 'pending' } }, 201);
});

// — Curator: get my pending approvals (waiting list) —
lessonApprovals.get('/my', requireRole('teacher'), async (c) => {
  const user = c.get('user');
  const rows = await c.env.DB.prepare(
    `SELECT la.*, ts.topic, ts.planned_date
    FROM lesson_approvals la
    JOIN tarbie_sessions ts ON ts.id = la.session_id
    WHERE la.curator_id = ?
    ORDER BY la.created_at DESC`
  ).bind(user.id).all();

  return c.json({ success: true, data: rows.results });
});

// — Admin: get all pending approvals for school —
lessonApprovals.get('/pending', requireRole('admin'), async (c) => {
  const user = c.get('user');
  const rows = await c.env.DB.prepare(
    `SELECT la.*, ts.topic, ts.planned_date, u.full_name as curator_name
    FROM lesson_approvals la
    JOIN tarbie_sessions ts ON ts.id = la.session_id
    JOIN users u ON u.id = la.curator_id
    WHERE la.school_id = ? AND la.status = 'pending'
    ORDER BY la.created_at ASC`
  ).bind(user.school_id).all();

```

```

    return c.json({ success: true, data: rows.results });
  });

// — Admin: get all approvals for school (all statuses) —
lessonApprovals.get('/all', requireRole('admin'), async (c) => {
  const user = c.get('user');
  const rows = await c.env.DB.prepare(
    `SELECT la.*, ts.topic, ts.planned_date, u.full_name as curator_name
      FROM lesson_approvals la
      JOIN tarbie_sessions ts ON ts.id = la.session_id
      JOIN users u ON u.id = la.curator_id
      WHERE la.school_id = ?
      ORDER BY la.created_at DESC
      LIMIT 100`
  ).bind(user.school_id).all();

  return c.json({ success: true, data: rows.results });
});

// — Admin: approve lesson —
lessonApprovals.post('/:id/approve', requireRole('admin'), async (c) => {
  const user = c.get('user');
  const approvalId = c.req.param('id');

  const approval = await c.env.DB.prepare(
    `SELECT * FROM lesson_approvals WHERE id = ? AND school_id = ?`
  ).bind(approvalId, user.school_id).first<{ id: string; status: string; session_id:
string; curator_id: string }>();

  if (!approval) {
    return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Approval
not found' }, 404);
  }

  if (approval.status !== 'pending') {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Already
processed' }, 400);
  }

  // Get admin's signature
  const adminSig = await c.env.DB.prepare(
    `SELECT id FROM user_signatures WHERE user_id = ?`
  ).bind(user.id).first<{ id: string }>();

  const now = new Date().toISOString();

  await c.env.DB.prepare(
    `UPDATE lesson_approvals SET status = 'approved', approved_by = ?, approved_at = ?,
admin_signature_id = ?, updated_at = ? WHERE id = ?`
  ).bind(user.id, now, adminSig?.id || null, now, approvalId).run();

  // Update session status to 'planned'
  await c.env.DB.prepare(
    `UPDATE tarbie_sessions SET status = 'planned', updated_at = ? WHERE id = ?`
  ).bind(now, approval.session_id).run();

  // Send SESSION_PLANNED notification to students
  const session = await c.env.DB.prepare(
    `SELECT ts.topic, ts.planned_date, ts.class_id, ts.teacher_id, c.name as class_name
      FROM tarbie_sessions ts JOIN classes c ON c.id = ts.class_id WHERE ts.id = ?`
  ).bind(approval.session_id).first<{ topic: string; planned_date: string; class_id:
string; teacher_id: string; class_name: string }>();

  if (session) {

```

```

const students = await c.env.DB.prepare(
  'SELECT student_id FROM class_students WHERE class_id = ?'
).bind(session.class_id).all<{ student_id: string }>();
const userIds = [session.teacher_id, ...students.results.map(s => s.student_id)];
const queueMsg: QueueMessage = {
  event_type: 'SESSION_PLANNED',
  session_id: approval.session_id,
  user_ids: userIds,
  template_vars: { topic: session.topic, date: session.planned_date, class_name:
session.class_name, teacher_name: '' },
  attempt: 0,
};
await c.env.NOTIFICATION_QUEUE.send(queueMsg);
}

// Notify curator that lesson was approved
c.executionCtx.waitUntil(
  notifyCuratorApprovalResult(approval.curator_id, session?.topic ?? '', true, null,
c.env, c.env.TELEGRAM_BOT_TOKEN)
  .catch(() => {})
);

return c.json({ success: true, data: { id: approvalId, status: 'approved', approved_at:
now } });
});

// — Admin: reject lesson —
lessonApprovals.post('/:id/reject', requireRole('admin'), async (c) => {
  const user = c.get('user');
  const approvalId = c.req.param('id');
  const body = await c.req.json() as { comment?: string };

  const approval = await c.env.DB.prepare(
    'SELECT * FROM lesson_approvals WHERE id = ? AND school_id = ?'
  ).bind(approvalId, user.school_id).first<{ id: string; status: string }>();

  if (!approval) {
    return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Approval
not found' }, 404);
  }

  if (approval.status !== 'pending') {
    return c.json({ success: false, code: ERROR_CODES.VALIDATION_ERROR, message: 'Already
processed' }, 400);
  }

  const now = new Date().toISOString();

  const approval2 = await c.env.DB.prepare(
    'SELECT session_id, curator_id FROM lesson_approvals WHERE id = ?'
  ).bind(approvalId).first<{ session_id: string; curator_id: string }>();

  await c.env.DB.prepare(
    `UPDATE lesson_approvals SET status = 'rejected', approved_by = ?, admin_comment = ?,
updated_at = ? WHERE id = ?`
  ).bind(user.id, body.comment || null, now, approvalId).run();

  // Cancel the session
  await c.env.DB.prepare(
    `UPDATE tarbie_sessions SET status = 'cancelled', updated_at = ? WHERE id = ? AND
status = 'pending_approval'`
  ).bind(now, approval2?.session_id ?? '').run();

  // Notify curator via Telegram
  if (approval2) {

```



```

        const sess = await c.env.DB.prepare('SELECT topic FROM tarbie_sessions WHERE id = ?')
        .bind(approval2.session_id).first<{ topic: string }>();
        c.executionCtx.waitUntil(
            notifyCuratorApprovalResult(approval2.curator_id, sess?.topic ?? '', false,
            body.comment || null, c.env, c.env.TELEGRAM_BOT_TOKEN)
        ).catch(() => {});
    });
}

return c.json({ success: true, data: { id: approvalId, status: 'rejected' } });
});

// — Get approval document data (for PDF generation) —
lessonApprovals.get('/:id/document', async (c) => {
    const user = c.get('user');
    const approvalId = c.req.param('id');

    const approval = await c.env.DB.prepare(
        `SELECT la.*, ts.topic, ts.planned_date,
        cu.full_name as curator_name,
        au.full_name as admin_name
        FROM lesson_approvals la
        JOIN tarbie_sessions ts ON ts.id = la.session_id
        JOIN users cu ON cu.id = la.curator_id
        LEFT JOIN users au ON au.id = la.approved_by
        WHERE la.id = ? AND la.school_id = ?`
    ).bind(approvalId, user.school_id).first<{
        id: string; status: string; curator_name: string; admin_name: string;
        topic: string; planned_date: string; approved_at: string;
        curator_signature_id: string; admin_signature_id: string;
        curator_id: string; approved_by: string;
    }>();

    if (!approval) {
        return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Not
found' }, 404);
    }

    // Get signatures (only if approved; for pending - return nulls)
    let curatorSignature: string | null = null;
    let adminSignature: string | null = null;
    let adminSignature2: string | null = null;

    if (approval.curator_id) {
        const cs = await c.env.DB.prepare(
            'SELECT signature_data FROM user_signatures WHERE user_id = ?'
        ).bind(approval.curator_id).first<{ signature_data: string }>();
        curatorSignature = cs?.signature_data || null;
    }

    if (approval.status === 'approved' && approval.approved_by) {
        const as2 = await c.env.DB.prepare(
            'SELECT signature_data, signature_data_2 FROM user_signatures WHERE user_id = ?'
        ).bind(approval.approved_by).first<{ signature_data: string; signature_data_2: string
| null }>();
        adminSignature = as2?.signature_data || null;
        adminSignature2 = as2?.signature_data_2 || null;
    }

    return c.json({
        success: true,
        data: {
            id: approval.id,
            topic: approval.topic,
            planned_date: approval.planned_date,
            curator_name: approval.curator_name,

```

```

        admin_name: approval.admin_name,
        approved_at: approval.approved_at,
        curator_signature: curatorSignature,
        admin_signature: adminSignature,
        admin_signature_2: adminSignature2,
        status: approval.status,
    },
    });
});
});

// — Download the uploaded Word file —
lessonApprovals.get('/:id/file', async (c) => {
    const user = c.get('user');
    const approvalId = c.req.param('id');

    const approval = await c.env.DB.prepare(
        'SELECT word_file_url, word_file_name, school_id, curator_id FROM lesson_approvals
        WHERE id = ?'
    ).bind(approvalId).first<{ word_file_url: string; word_file_name: string; school_id:
    string; curator_id: string }>();

    if (!approval || approval.school_id !== user.school_id) {
        return c.json({ success: false, code: ERROR_CODES.USER_NOT_FOUND, message: 'Not
        found' }, 404);
    }

    const base64Value = await c.env.KV.get(approval.word_file_url, { type: 'text' });
    if (!base64Value) {
        return c.json({ success: false, message: 'File not found in storage' }, 404);
    }

    // Decode base64 back to binary
    const binaryStr = atob(base64Value);
    const bytes = new Uint8Array(binaryStr.length);
    for (let i = 0; i < binaryStr.length; i++) {
        bytes[i] = binaryStr.charCodeAt(i);
    }

    return new Response(bytes, {
        status: 200,
        headers: {
            'Content-Type': 'application/vnd.openxmlformats-
            officedocument.wordprocessingml.document',
            'Content-Disposition': `attachment; filename="$
            {encodeURIComponent(approval.word_file_name)}"`,
            'Content-Length': String(bytes.length),
            'Access-Control-Allow-Origin': c.req.header('Origin') || '*',
            'Access-Control-Allow-Credentials': 'true',
        },
    });
});

export { lessonApprovals };

```

Қосымша И

Суреттер мен скриншоттарды орналастыру картасы

Бұл қосымша – дайын құжатқа суреттер мен скриншоттарды дұрыс қоюға арналған нұсқаулық. Құжатта әрбір сурет орны «[ОСЫ ЖЕРГЕ СУРЕТ / СКРИНШОТ ҚОЙЫҢЫЗ]» деген сұр жолақпен белгіленген. Төмендегі кесте бойынша әр суретті тиісті тармаққа қойыңыз. «Түрі» бағанында: «Диаграмма» – өзіңіз сызатын схема (draw.io, dbdiagram.io), «Скриншот» – жүйеден түсірілетін экран суреті.

Кесте И.1 Суреттерді орналастыру картасы

Сурет №	Түрі	Не қою керек	Қай тармаққа	Қайдан алу / түсіру
2.1-сурет	Диаграмма	Жүйенің жалпы архитектурасы	2.3 тармақ	draw.io / Excalidraw-та схема сызу
2.2-сурет	Диаграмма	Use-case (пайдалану нұсқалары) диаграммасы	2.4 тармақ	draw.io / PlantUML-да диаграмма сызу
2.3-сурет	Диаграмма	Деректер қорының ER-диаграммасы	2.5 тармақ	dbdiagram.io немесе DBever-дан экспорт
2.4-сурет	Диаграмма	Сабақты бекіту алгоритмінің блок-схемасы	2.8 тармақ	draw.io-да блок-схема сызу
2.5-сурет	Скриншот	Жүйеге кіру (авторизация) беті	2.11 тармақ	Қосымшаны браузерде ашып /login бетін түсіру
2.6-сурет	Скриншот	Әкімшінің басты беті (dashboard)	2.11 тармақ	Әкімші болып кіріп, басты панельді түсіру
2.7-сурет	Скриншот	Тәрбие сағаттарының тізімі	2.11 тармақ	Куратор болып «Тәрбие сағаттары» бетін түсіру
2.8-сурет	Скриншот	Тәрбие сағатын құру формасы	2.11 тармақ	«Жаңа тәрбие сағаты» формасын ашып түсіру
2.9-сурет	Скриншот	Қатысуды белгілеу терезесі	2.11 тармақ	Сабақты ашып, қатысу терезесін түсіру
2.10-сурет	Скриншот	Құжатты бекіту және қолтаңба алаңы	2.11 тармақ	«Сабақты бекіту» бетін және қолтаңба терезесін түсіру
2.11-сурет	Скриншот	Студенттің рейтинг беру терезесі	2.11 тармақ	Студент болып кіріп, баға беру терезесін түсіру
2.12-сурет	Скриншот	Telegram боттың сұхбат терезесі	2.11 тармақ	Telegram-да ботпен сұхбатты (/start) түсіру
2.13-сурет	Скриншот	Автоматты тестілеу нәтижелері	2.11 тармақ	Тестілеу панелін / Playwright есебін түсіру

Ескерту. Сурет қойғаннан кейін «[ОСЫ ЖЕРГЕ СУРЕТ ...]» деген сұр мәтінді өшіріп, оның орнына суретті кірістіріңіз. Суреттің астындағы «N-сурет – атауы» жазуы өзгеріссіз қалуы тиіс. Барлық суреттер тармақтың мәтінінде сілтеме жасалған ретпен орналасады.